Prague University of Economics and Business
Faculty of Informatics and Statistics

A Data Integration Framework for Enterprise Application Security

MASTER'S THESIS

Study program: Applied Data Analytics and Artificial Intelligence
Specialization: Retail and E-Commerce Data Analytics

Author: Bc. Vojtěch Kraus
Supervisor: Ing. Aleš Kotuč

Prague, May 2026

Declaration on the Use of Artificial Intelligence

I confirm that artificial intelligence tools were used in the preparation of the submitted work. Specifically, they were used in the following ways:

- improvement of grammar, language style, and conciseness of the thesis, including review cycles that shortened the text by rephrasing longer passages and relocating details into the attached product documentation,
- generation of source code for figures in the thesis,
- generation of source code attached to the thesis (`mvp.zip`),
- generation of product documentation attached to the thesis (`docs.zip`),
- assistance with the LaTeX build scripts.

Each use of any of the above methods is separately documented in an appendix to the thesis, which provides a more detailed explanation of which tool was used in which specific part of the work; the author's own contribution is highlighted.

I confirm that:

- my work is original and was prepared in accordance with ethical standards, in particular the Code of Ethics of the Prague University of Economics and Business; that all information sources used are properly cited in the work; and that any generated content has been verified by me;
- I accept full responsibility for the entire content of the thesis.

Acknowledgements

I would like to thank Ing. Aleš Kotuč for his guidance and valuable feedback.

Abstract

To protect their business-critical applications from cybersecurity threats, enterprise security teams rely on numerous tools to detect and resolve vulnerabilities in application source code, configuration, dependencies, CI/CD pipelines, and runtime infrastructure. These tools use different integration patterns, protocols, and data formats, making it difficult to consistently parse, deduplicate, triage, and group their findings. In large environments, additional challenges emerge when linking all findings to the corresponding business applications. This thesis presents a data framework to tackle those challenges, delivering three distinct contributions: (1) a requirements specification for an application security data platform, (2) a reusable and extensible design covering architecture, data model, and medallion-based data pipeline, and (3) a reference implementation built on the Databricks platform. Together, these contributions offer application security teams a foundation for building a robust data platform independent of individual cybersecurity vendors, relieving them of risks usually associated with vendor lock-in.

Keywords

application security, software security, DevSecOps, security integration, data consolidation, medallion architecture, Databricks

Contents

List of Figures

List of Tables

List of source codes

Introduction

Motivation

To protect business critical applications, security teams rely on a large variety of tools: Static Application Security Testing (SAST) scanners analyze source code, Software Composition Analysis (SCA) tools check software dependencies, Dynamic Application Security Testing (DAST) probes running applications, and secret scanners flag exposed credentials in code (Chess & West, 2007; Howard & LeBlanc, 2006; Stuttard & Pinto, 2011).

Further detection runs against build and deployment pipelines, which face their own classes of threats such as malicious dependencies published to public package registries, compromised build agents injecting code into release artifacts, tampered container base images, or leaked Continuous Integration and Continuous Delivery (CI/CD) credentials used to push unauthorized changes (Ohm et al., 2020; OpenSSF, 2024). Security teams also deploy Integrated Development Environment (IDE) plugins and security context tools integrated into Artificial Intelligence (AI) coding assistants such as Claude Code or GitHub Copilot to catch problems before code reaches version control (GitHub, 2025a; Semgrep, 2024; Snyk, 2024).

In large enterprises, dozens of such tools produce findings through different data formats and integration patterns (Anderson, 2020; Stallings & Brown, 2024). As application security architect in a large organization with thousands of developers and tens of thousands of repositories, I have seen firsthand how difficult such environment is to secure. Two types of difficulty stand out.

The first is **detection diversity**. Tools in the same category scan differently and produce different results, so a single tool is rarely sufficient (Chess & West, 2007; McGraw, 2006). Running multiple tools in parallel improves coverage but creates the opposite problem: the same vulnerability is reported several times, and findings must be deduplicated before they can be triaged or counted (Gardner et al., 2023; Jacobs et al., 2020).

The second is **data consolidation**. Findings lack the business context needed for prioritization: the owning application, its criticality tier, and its regulatory scope live in a separate Configuration Management Database (CMDB), not in the security tool (AXELOS, 2019; Williams & Gonzalez, 2021). Correlation within the findings themselves is nontrivial: SAST results point to a repository, file, and line, while DAST results point to a Uniform Resource Locator (URL) or host, so cross tool correlation requires additional deployment and inventory metadata (Chess & West, 2007; Stuttard & Pinto, 2011). Integration patterns compound the problem. Push based models, where scanners upload reports into a vulnerability management interface, fragment data and do not scale across dozens of tools, and pull based Application Programming Interface (API) connectors, where they exist, are usually gated

behind commercial tiers (DefectDojo, 2024; Gardner et al., 2023).

Consolidating application security data is a data engineering problem. Industry addresses it with commercial Application Security Posture Management (ASPM) products offering dashboards and aggregated views (Gardner et al., 2023), but these are proprietary and inflexible. They create vendor lockin and prevent security teams from customizing pipelines, applying their own Machine Learning (ML) models, or integrating with internal systems.

The most notable open source alternative, DefectDojo, is a Django based vulnerability management web application (DefectDojo, 2024), not a data first platform. Its relational backend limits scale, and ingestion relies mostly on push based integrations or manual uploads. Pull based API connectors exist only in the commercial DefectDojo Pro tier.

Databricks Lakewatch, announced in March 2026, is a lakehouse native Security Information and Event Management (SIEM) (Databricks, 2026a). It runs on the same lakehouse platform this thesis adopts but addresses a different domain: runtime threat detection, alert triage, and log analytics, not application security posture. The distinction is discussed in Section 1.8.2.

What is missing is a vendor agnostic framework that approaches application security as a data engineering problem, providing a reusable architecture for ingestion, normalization, deduplication, and serving at enterprise scale. The pace of tool adoption outstrips the engineering effort available to write connectors for each one. A framework that encodes integration logic declaratively, as schemas, a connector contract, and a catalog of skills an AI coding agent can execute, **should reduce onboarding a new source to a five step procedure** (Section 3.1) rather than a pipeline redesign, with the cost per source bounded by the cost of invoking the documented skills under supervision. Step three of the procedure (apply the mapping declaratively) is currently a specification thread, not an executed step in the Minimum Viable Product (MVP), which still builds the silver DataFrame in Python. Future Work in the Conclusion records the declarative applicator as the consolidation that closes this gap. Chapter 3 and Section 3.6.3 examine how far this reduction holds under the empirical methodology actually exercised.

This raises the central research question: **how can diverse enterprise application security findings be consolidated into a unified, vendor agnostic data platform that serves the analytical and operational needs of security teams?**

Objective

The main goal is to specify, design, and implement a data integration framework for enterprise application security that consolidates findings from tools into a unified platform, enabling consistent normalization, deduplication, triage, and correlation across business applications.

The subgoals are:

1. **Analysis** (Chapter 1): Survey the application security domain and its adjacent data sources. Characterize asset inventories, software development platforms, static and dynamic security tooling, and the relevant data engineering patterns. Review related work, identify the research gap, and select source systems to carry forward. The API analysis per source grounding these findings is collected on the external documentation site.
2. **Framework** (Chapter 2): Formalize the requirements specification and propose a reusable blueprint covering solution architecture, a medallion based data model (bronze, silver, gold) (Strengholt, 2025), a connector framework, and an analytics and serving framework. The blueprint targets the Databricks lakehouse (Databricks, 2025c, 2025e; Lee et al., 2024) while separating general patterns from platform specific choices. The technology rationale is given in Section 2.1.1.
3. **Implementation** (Chapter 3): Produce a reference implementation on Databricks that instantiates the framework for the nine selected sources, all of which ship working ingest and transform modules. Several open items remain at the surrounding orchestration layer: notebook entry wrappers for one connector and the analytics layer scaffolding. The traceability matrix at platform/reference/catalog records PASS or N/A for every (requirement $\times$ source) cell, and the Limitations section of the Conclusion enumerates the open items. The deliverable is a minimum viable product on a sample chosen to cover the ingestion and integration patterns the framework must support, not an exhaustive integration of every security tool. Validation runs through automated tests with requirement traceability and Lakeflow Declarative Pipelines (LDP) data quality expectations.

The scope is application security posture across three tiers of detection. **Static testing** covers preexecution analysis of source, dependencies, configuration, and build artifacts: SAST, SCA, secret scanning, container and Infrastructure as Code (IaC) scanning, and supply chain integrity checks. **Dynamic testing** covers active probing of running applications: DAST (OWASP ZAP (OWASP Foundation, 2024d)) and penetration testing. **Runtime security** covers observational telemetry from production systems. One representative source (AWS WAF sampled requests (Amazon Web Services, 2026)) demonstrates that the correlation model links runtime findings to applications via deployment metadata. Broader runtime operations (threat detection, incident response, log analytics) are out of scope.

Chapter 1 and Chapter 2 are framed against the full ASPM scope. Chapter 3 instantiates the framework against the nine sources selected in Section 1.9. Onboarding a tenth source repeats the procedure described in Section 3.1 without changes to the framework.

The intended audience is application security architects and data platform engineers who need to consolidate security findings at scale without vendor lockin.

Methodology

This thesis follows the design science research methodology of Hevner et al. (2004) and Peffers et al. (2007) in three phases aligned with the main chapters. The **analysis** phase combines a literature review with a systematic study of source systems and identifies the research gap. The **framework** phase derives requirements and designs the architecture, data model, connector contract, and analytics patterns. The **implementation** phase targets a reference implementation with an AI instantiable integration layer: a chain of four skills (analyze-source, provision-source, generate-connector, validate-implementation) was authored from the connector contract and run end to end against all nine sources under reviewer subagent supervision, producing every per-source connector greenfield. Section 3.6.3 grades the outcome.

The thesis delivers three contributions: a **requirements specification**, a **reusable framework** covering architecture, data model, connector contract, and analytics and serving patterns, and a **reference implementation** on Databricks. The integration layer of the reference implementation is AI **instantiable** under skill catalog invocation, defended in Section 3.6.3.

Structure

Chapter 1 surveys the problem domain and closes with related work, gap analysis, and source selection. Chapter 2 presents the framework: solution architecture, data model, connector framework, and analytics and serving framework. Chapter 3 reports the MVP reference implementation against the nine selected sources. The Conclusion evaluates outcomes against the objectives, defends the AI instantiability claim in Section 3.6.3, discusses limitations, and suggests future work. Appendix A discloses the use of generative AI tooling. Figure 1 summarizes the contribution flow.

External Documentation Site

The full requirements specification is published at `https://vkraus.github.io/appsec-mvp/` instead of inline. It is structured into four sections (Requirements, Functional Specification, Design, Tests), generated with MkDocs Material from `mkdocs/docs/` in the `appsec-mvp` repository, and deployed via GitHub Pages. Where the main chapters name a page, they link to it directly. The relevant top level sections are the per category capability matrix, the documented mapping requirements, the connectors reference hub, and the requirement catalog and traceability matrix.

Offline copies of the reference implementation and the documentation site are at-

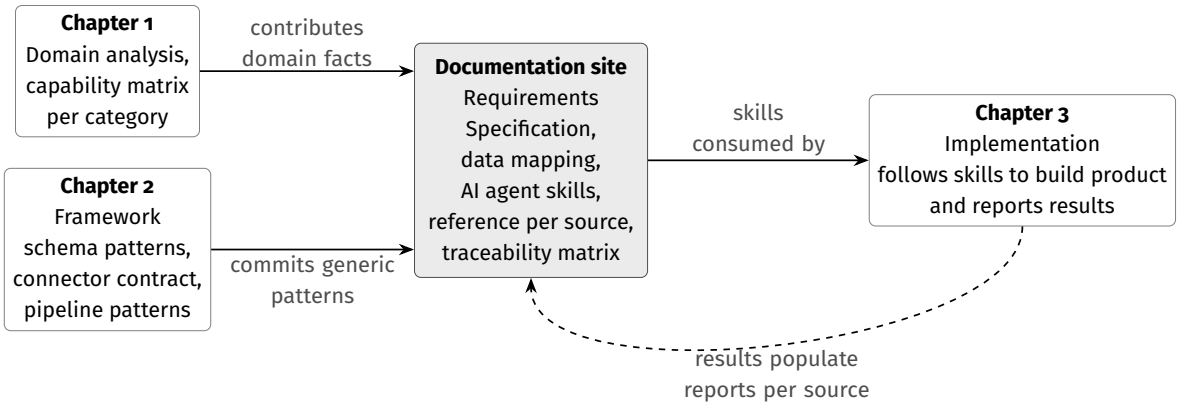

Figure 1

Thesis contribution flow. Chapter 1 and Chapter 2 contribute to the Requirements Specification published at `https://vkraus.github.io/appsec-mvp/`*. Chapter 3 consumes the specification, with reviewer subagent cycles catching defect classes evaluated in Section 3.6.3. Implementation results populate the per source Implementation reports on the site (dashed return arrow).*

tached as `mvp.zip` and `docs.zip` (the latter includes a prerendered static site under `docs/site/index.html`). Both are snapshots at the commit from which this Portable Document Format (PDF) was built.

1. Analysis

This chapter analyzes the functional domains relevant to building an application security data platform. Each section combines theoretical foundations with practical tool and API analysis, focusing on source system interfaces, data models, and integration patterns. Technology choices are deferred to Chapter 2 and Chapter 3.

1.1 Asset Discovery and Inventory

Asset discovery and inventory is the foundational data source for any application security data platform. Organizations maintain application inventories in a CMDB or custom catalog, while cloud providers and container orchestrators expose asset inventories through APIs. Without this context, a vulnerability cannot be attributed to an owner, scored for business impact, or prioritized.

1.1.1 Application Portfolio Management

Enterprise organizations maintain catalogs of business applications with metadata such as name, description, ownership, lifecycle status, and relationships to other assets (AXELOS, 2019). These registries also carry criticality tiers (Tier 1 mission critical, Tier 2 important, Tier 3 noncritical) derived from business impact assessments along the Confidentiality, Integrity, and Availability (CIA) triad (National Institute of Standards and Technology, 2004; Stallings & Brown, 2024) and regulatory scope flags.

The attribute that matters most for data integration is the mapping from business applications to their technical assets: a single application may span repositories, microservices, infrastructure, and deployment environments. When the framework ingests a vulnerability from a SAST scanner linked to a repository, this mapping determines which application is affected, who owns it, and how critical it is.

1.1.2 Configuration Management Databases

ServiceNow is widely deployed as a CMDB platform in enterprise environments and leads the global IT Service Management (ITSM) market with over 44% share (Gartner, 2024), the closest available proxy for CMDB adoption. It serves as the reference platform for this thesis. A CMDB organizes all managed assets as Configuration Items (CIs), each with attributes describing its type, status, and ownership (AXELOS, 2019).

ServiceNow arranges CIs in a class hierarchy rooted at the `cmdb_ci` base table. Each subclass inherits base attributes (name, `sys_id`, operational status, environment) and adds domain specific fields. For application inventory purposes, the most relevant class is `cmdb_ci_business_app` (Business Application), which extends the base CI with attributes such as version, business unit, used for classification, and references to the owning support group (ServiceNow, 2024a). The MVP pipeline ingests this class together with `cmdb_rel_ci`.

Beyond the application record itself, the CMDB captures relationships between CIs through a dedicated relationship table (`cmdb_rel_ci`). These model dependencies such as "runs on" (application to server), "depends on" (application to database), and "hosted on" (application to cloud service). The most valuable relationships for the framework are those linking business applications to technical assets corresponding to entities in the security data: repository names, deployment targets, and infrastructure components. ServiceNow allows custom attributes and relationship types, so available fields vary between deployments.

1.1.3 Integration Options

Integrating CMDB data requires choosing between two strategies: pulling data from ServiceNow via its APIs, or pushing data from a custom ServiceNow application to an external target.

1.1.3.1 ServiceNow APIs

ServiceNow exposes two Representational State Transfer (REST) APIs for CMDB extraction. The Table API (`/api/now/table/{tableName}`) provides generic queries against any table with server side filtering, field selection, and pagination (ServiceNow, 2024d). The CMDB Instance API (`/now/cmdb/instance/{className}`) understands the CI class hierarchy and retrieves a CI with its relationships in a single call (ServiceNow, 2024a). Authentication is Basic auth with a service account or OAuth 2.0 (ServiceNow, 2024b), and the `sys_updated_on` timestamp serves as a reliable high water mark for incremental extraction.

1.1.3.2 ServiceNow Application

An alternative is to push from the source. A scoped ServiceNow application can use Outbound REST Messages on a schedule, or expose a Scripted REST API (ServiceNow, 2024c) that prejoins application records with relationships in one response. The advantage is full access to ServiceNow scripting and relationship traversal. The disadvantage is that development and maintenance live inside ServiceNow, with schema changes coordinated across two platforms. This approach suits organizations with mature ServiceNow development teams.

1.1.3.3 Integration Challenges

The application inventory is the least standardized integration among all data sources. Security testing tools converge around common API patterns and formats, but each organization structures its application inventory differently: hierarchy, granularity of asset mapping, and captured attributes vary substantially (AXELOS, 2019). This makes the inventory connector hardest to generalize and most likely to require customization.

Data quality in CMDBs is a persistent concern (Williams & Gonzalez, 2021). Records are often incomplete or stale. A common case is a decommissioned application still listed as active with no current owner. Integrations must account for these issues through validation rules and defaults that flag problems without blocking the pipeline.

Despite these difficulties, the application inventory is indispensable: without its business context, findings lack organizational meaning for prioritization and governance.

Alternative platforms include BMC Helix, Device42, and the open source Ralph. Cloud native services such as AWS Config and Azure Resource Graph expose equivalent inventories for cloud resources, consumed through provider native APIs with IAM scoped credentials. The bronze ingestion pattern (Chapter 2) applies to any source with an inventory API.

1.2 Software Development

Repositories, build pipelines, and issue trackers form the second major data source category (Pressman & Maxim, 2019; Sommerville, 2015). These platforms share a common integration pattern: REST over JavaScript Object Notation (JSON), authentication via OAuth tokens, Personal Access Tokens (PATs), or service accounts, page or cursor pagination, and rate limits that require backoff. This consistency enables shared connector logic.

1.2.1 Source Code Management

For this thesis, the repository is the primary entity around which findings are grouped (Chacon & Straub, 2014; Skoulikari, 2024; Winters et al., 2020).

Development teams host Git repositories on cloud Source Code Management (SCM) platforms. GitHub is dominant (81% adoption for code collaboration), followed by GitLab (36%), Bitbucket, and Azure DevOps (GitHub, 2025b; Stack Overflow, 2025). Most large organizations run several platforms, so multi platform ingestion is a practical requirement.

Beyond source code, SCM platforms expose security relevant metadata: primary language, last activity, visibility, archive status. Branch protection rules indicate development process

maturity. Team and contributor assignments establish ownership feeding into the mapping from application to team in Section 1.1.

These platforms share three API patterns: token based authentication (OAuth or PATs), page or cursor pagination, and rate limits that require backoff. GitHub also exposes a GraphQL endpoint alongside its REST API (v3), with a 5 000 requests per hour limit (GitHub, 2024a, 2024b). GitLab, Bitbucket, and Azure DevOps Repos provide comparable interfaces.

1.2.2 Continuous Integration and Delivery

Application security is fundamentally about securing the Software Development Lifecycle (SDLC): CI/CD pipelines are the spine of the modern SDLC and each stage can integrate security tools (Forsgren et al., 2018). The dominant platforms are GitHub Actions, Jenkins, GitLab CI, and Azure Pipelines (JetBrains, 2025). Many organizations run more than one, so the multi platform pattern from SCM repeats here.

CI/CD platforms provide contextual data about security scans. Pipeline definitions reveal which tools run. Build results show whether gates passed. Execution metadata records commit, timing, and duration. A repository whose last scan ran three months ago, or whose scans consistently fail, signals coverage gaps and tool health. This is operational context rather than a primary finding source.

Integration patterns vary across CI/CD platforms more than across SCM: GitHub Actions and Azure Pipelines expose REST APIs, Jenkins uses an Extensible Markup Language (XML) based API, and GitLab CI piggybacks on the GitLab API. Retrieved data is structurally similar across them: pipeline identifiers, run statuses, timestamps, and commit references. None of these is built as a connector in the reference implementation.

1.2.3 Issue Tracking

Issue trackers record remediation status. When a finding is assigned for remediation, an issue is created in the tracker for the team manually or through automation. The framework reads issue status to determine whether a finding is under active remediation, who is responsible, and whether it has been resolved within the Service Level Agreement (SLA) window.

Jira is the dominant enterprise issue tracker. The 2025 Stack Overflow Developer Survey reports Jira at 46% adoption, behind only GitHub (81%) and ahead of GitLab (36%) (Stack Overflow, 2025). Azure DevOps Work Items serves the Microsoft ecosystem. GitHub Issues and GitLab Issues provide lightweight built in tracking tied to their SCM platforms. Many organizations use multiple trackers across teams.

These platforms expose REST APIs with JSON responses and paginated results, and most support webhooks for real time status change notifications. None is built as a connector in the reference implementation.

Integration is bidirectional: the framework reads remediation status and may create new issues when findings require attention. This raises idempotency concerns (avoiding duplicate issue creation) and state synchronization (keeping the view of the framework consistent with the tracker).

1.3 Static Application Security

Application security covers practices and tools for identifying vulnerabilities throughout the software lifecycle (Anderson, 2020; Stallings & Brown, 2024). McGraw (2006) advocated for security touchpoints spanning development. The DevSecOps approach (Kim et al., 2021; Mack, 2023) realizes this by embedding security testing into CI/CD pipelines. SAST examines source code, SCA checks third party dependencies, secret scanners flag leaked credentials, and DAST probes running applications. These tools produce large volumes of data in different APIs, formats, and severity models. The Open Worldwide Application Security Project (OWASP) Top 10 classifies common web application risks (OWASP Foundation, 2021) but does not define a data interchange standard.

Several cross cutting standards bring partial consistency. The Common Vulnerabilities and Exposures (CVE) system assigns unique identifiers to known vulnerabilities (MITRE Corporation, 2024a). Common Weakness Enumeration (CWE) classifies weakness types (MITRE Corporation, 2024b). Common Vulnerability Scoring System (CVSS) scores attack vector, complexity, and impact on a 0–10 scale (Forum of Incident Response and Security Teams, 2019). Exploit Prediction Scoring System (EPSS) adds a predictive signal (Jacobs et al., 2020): the ML computed probability that a vulnerability will be exploited within 30 days. Cybersecurity and Infrastructure Security Agency (CISA) maintains the Known Exploited Vulnerabilities (KEV) catalog (Cybersecurity and Infrastructure Security Agency, 2024), which adds a third signal: confirmed active exploitation from real world incidents.

For data interchange, Static Analysis Results Interchange Format (SARIF) defines a JSON based format for static analysis results (OASIS Open, 2024). CycloneDX and Software Package Data Exchange (SPDX) provide Software Bill of Materials (SBOM) schemas (Linux Foundation, 2024; OWASP Foundation, 2024a). Open Cybersecurity Schema Framework (OCSF) standardizes security event exchange for SIEM and Security Orchestration, Automation, and Response (SOAR) use cases (OCSF Project, 2024). No single format covers all tool categories:

- SARIF addresses static analysis but not SCA or runtime findings.
- CycloneDX and SPDX cover software composition, not code level vulnerabilities.
- OCSF serves security operations, not application security posture.

This gap motivates the normalization model in Chapter 2.

Static analysis tools examine source code and build artifacts without executing the application. Their findings point to repositories, files, and lines, so developers own them directly. Mobile application security testing is excluded, as the framework targets web applications and backend services.

1.3.1 Static Application Security Testing

SAST examines source code, bytecode, or binary code without executing the program. Chess and West (2007) cover taint analysis, control flow analysis, and pattern matching for injection flaws, buffer overflows, and insecure data handling. Broader rule sets catch more issues and generate more noise. That is the central SAST tradeoff.

Server tools (SonarQube, Checkmarx, Fortify) expose a REST API. Command Line Interface (CLI) tools (Semgrep) produce JSON or SARIF files. Platform integrated scanners (GitHub Code Scanning, GitLab SAST) report through the host SCM API. When a platform scanner and a standalone tool both scan a repository, the overlap requires deduplication.

Findings include a rule identifier, file path and line number, severity, code snippet, and remediation guidance. Severity models differ: SonarQube uses five levels (blocker, critical, major, minor, info), Semgrep and Checkmarx use three (high, medium, low). Severity normalization is therefore a core transformation.

1.3.2 Software Composition Analysis

SCA identifies known vulnerabilities in third party dependencies. Modern applications pull in hundreds of transitive dependencies (Anderson, 2020), any of which can introduce a vulnerability. Ohm et al. (2020) cover supply chain attacks including typosquatting, dependency confusion, and malicious package injection. SCA tools analyze dependency manifests, resolve the full tree, and cross reference versions against vulnerability databases such as the National Vulnerability Database (NVD).

Tools fall into three groups by integration: server platforms (OWASP Dependency-Track, Snyk) with a REST API, platform integrated scanners (Dependabot through GitHub GraphQL, GitLab dependency scanning) reached through the host API, and CLI alternatives such as OWASP Dependency-Check that write JSON or XML reports from CI/CD runs. Many tools also generate SBOMs in CycloneDX or SPDX format (Linux Foundation, 2024; OWASP Foundation, 2024a), increasingly required by regulators (National Telecommunications and Information Administration, 2021).

SCA output is more standardized than other categories because the underlying data comes from shared public databases (CVE, NVD). Findings include the vulnerable dependency

name and version, a CVE identifier, a CVSS score, the fixed version when available, and exploitability metadata. Tools may resolve the same dependency tree differently, so two scanners flag inconsistent CVEs. The framework handles this during deduplication: the same CVE reported by multiple tools against the same repository likely represents a single issue. The Supply-chain Levels for Software Artifacts (SLSA) framework (OpenSSF, 2024) complements SCA by verifying the integrity and provenance of build artifacts.

1.3.3 Secret Detection

Secret detection tools scan version control history for credentials, API keys, tokens, and other sensitive material. Once committed, a secret persists in Git history even after deletion from the working tree. Two techniques apply: regex patterns against known credential formats, and entropy scoring against unusually random strings. The entropy approach catches novel formats at higher false positive rates.

CLI tools (TruffleHog, GitLeaks, detect-secrets) write JSON reports parsed as artifacts. Commercial servers (GitGuardian) expose a REST API. Platform integrated scanners (GitHub Secret Scanning, GitLab Secret Detection) report through the host API (GitHub, 2024a) but only cover repositories on their own platform. There is no standard output format. Each tool defines its own JSON schema, none supports SARIF. Findings include secret type, file path, commit reference, and sometimes validity status, but field names and structures vary.

1.4 Dynamic Application Security Testing

DAST tests running applications by sending crafted Hypertext Transfer Protocol (HTTP) requests and analyzing responses for vulnerability indicators (Stuttard & Pinto, 2011). It requires a deployed application and detects vulnerabilities that appear only at runtime, such as authentication bypass, server misconfiguration, and injection flaws. This section covers active scanning. Section 1.5 covers passive telemetry.

The representative tools are OWASP Zed Attack Proxy (ZAP) (open source) and Burp Suite Enterprise, both exposing a REST API for scan management and alert retrieval. Findings include the target URL, affected HTTP parameter, vulnerability type mapped to CWE (MITRE Corporation, 2024b), exploitation evidence, and confidence rating. The core integration challenge is that DAST findings are URL based, not code based. Mapping them to source repositories requires deployment metadata or inventory data from Section 1.1. Without that link, DAST findings can be attributed to applications but not to teams or code locations.

1.5 Runtime Security

Runtime security covers passive telemetry from production workloads, sitting alongside the active scanning in Section 1.4 and completing the three detection tiers the framework uses: static, dynamic, and runtime. The representative tool in this tier is the Web Application Firewall (WAF). It does not execute scans. It inspects live HTTP traffic and emits events when rules fire, marking exploitation attempts against a workload or endpoint and linking to applications through deployment metadata instead of source code coordinates. Broader runtime operations such as SIEM driven incident response and general log analytics are adjacent but out of scope. The framework treats runtime security as one input tier among three, correlated with static and dynamic findings through the shared application inventory.

1.5.1 Web Application Firewalls

WAFs operate at the network perimeter, inspecting inbound HTTP traffic against rule sets such as the OWASP Core Rule Set (OWASP Foundation, 2024c). They log blocked and flagged requests with matched rule, request details, and action taken. Amazon Web Services (AWS) WAF delivers full request logs to Amazon S3 through Kinesis Data Firehose, which the framework consumes as the primary path. The `GetSampledRequests` API returns only a sampled subset and serves as a fallback (source reference). Cloudflare and Akamai expose comparable REST APIs for log retrieval and configuration.

These platforms bundle Distributed Denial of Service (DDoS) protection alongside WAF capabilities. DDoS mitigation logs traffic patterns, attack signatures, and mitigation actions, providing availability impact data that complements vulnerability findings.

WAFs emit append only event streams. The framework projects each event as one finding row on `silver.findings`, with severity derived from the WAF action and status fixed to `open`.

1.6 Source Integration Summary

The source characteristics reference page tabulates integration characteristics across the AppSec sources surveyed for the thesis.

Several patterns emerge. REST APIs with JSON responses are dominant, but XML, GraphQL, CLI output parsing, and manual uploads are all necessary. Standardization varies: SCA benefits from the CVE/NVD ecosystem, SAST is partially standardized through SARIF, and secret scanning has no standard format. Update frequencies range from continuous dependency monitoring to periodic penetration tests. These characteristics define what the ingestion layer in Chapter 2 must accommodate.

1.7 Data Engineering

Consolidating the diverse sources analyzed above into a unified analytical platform is a data integration problem. This section surveys the architectural paradigms and engineering patterns relevant to solving it.

1.7.1 Data Platform Architecture

Data warehouses (Inmon, 2005; Kimball & Ross, 2013) assume structured, stable sources, and security tool output is neither. Data lakes (Miloslavskaya & Tolstoy, 2016) accept any format through schema on read but risk "data swamps" without governance (DAMA International, 2017). The lakehouse (Armbrust et al., 2021) combines lake flexibility with warehouse governance through open table formats that add Atomicity, Consistency, Isolation, Durability (ACID) transactions, schema enforcement, and time travel on top of lake storage (Lee et al., 2024; Thalpati, 2024). It accepts semi structured tool output and gives the governance and fast queries needed for dashboards and operational lookups.

The medallion architecture (Strengholt, 2025) organizes lakehouse data into three layers:

- **Bronze**: Raw data with minimal transformation, preserving original source schemas.
- **Silver**: Cleaned, validated, and normalized data conforming to a unified schema.
- **Gold**: aggregated metrics and enriched datasets ready for queries.

This maps naturally to security data integration. Bronze captures raw tool output in source native schemas, silver normalizes findings into a vendor agnostic model, and gold computes aggregated risk metrics and enriched datasets stakeholders consume.

A security data platform must serve two access patterns. Online Analytical Processing (OLAP) queries power dashboards and trend analysis over large aggregated datasets. Online Transaction Processing (OLTP) access supports operational workflows such as issue tracker integration and real time risk lookups, where low latency reads and writes to individual records are essential. Both inform the serving architecture in Chapter 2.

1.7.2 Data Integration Patterns

Reis and Housley (2022) distinguish Extract, Transform, Load (ETL) from Extract, Load, Transform (ELT). ETL transforms data before loading, requiring upfront schema knowledge. ELT loads raw data first and transforms in place, aligning with lakehouse architectures where distributed compute engines perform transformations at scale (Thalpati, 2024). Security tool output varies and evolves, so upfront transformation breaks. ELT fits.

Full reingestion does not scale across thousands of repositories and dozens of tools.

Incremental ingestion pulls only new or changed records. The high water mark pattern tracks a timestamp or cursor per source. Change Data Capture (CDC) captures changes at the source level, propagating inserts, updates, and deletes as events (Reis & Housley, 2022).

Source APIs and formats change regularly, so rigid schemas break pipelines on every upstream change. Schema on read stores raw data in its original structure and applies schema at query time. Additive evolution accepts new fields (Lee et al., 2024), and existing queries keep working.

Reruns are idempotent so retries and replays do not duplicate. Each layer validates: schema at ingest, value ranges at normalization, referential integrity at aggregation. Records that fail validation go to quarantine tables (DAMA International, 2017).

Managed ingestion connectors complement hand coded integration. Databricks Lakeflow Connect ships first party managed connectors for ServiceNow, Salesforce, SQL Server, Workday, and Google Analytics (Databricks, 2025b), registered as Unity Catalog connections and consumed through a declarative pipeline definition that the platform schedules and incrementally refreshes. The Databricks Labs project Lakeflow Community Connectors (Databricks Labs, 2026) extends the catalogue with community maintained connectors built on the Spark Python Data Source API, including a connector for GitHub. Community connectors share the registration pattern of first party managed connectors but carry no compatibility guarantees and are pre release at the time of writing. The framework adopts managed and community connectors where their coverage and operational guarantees meet the connector contract; otherwise it falls back to a maintained Python Software Development Kit (SDK), then to data load tool (dlt), then to artifact path consumption.

1.7.3 Domain Data Model

The security domains analyzed above produce data about a common set of entities. A vendor agnostic conceptual model precedes physical schemas. Table 1.1 summarizes the entities and their role in the model.

The first five rows are primary entities. The rest add development and supply chain context. Figure 1.1 illustrates the relationships, with primary entities shown in bold.

The mapping from app to repo is the key relationship. It links findings to business context and is many to many: shared libraries serve multiple applications, and microservice applications span multiple repositories. Teams own applications, applications connect to repositories many to many, and findings are produced from pipeline runs against those repositories. Dependencies attach to repositories and reach vulnerabilities through SCA findings: a finding cites a specific library version and references a known CVE. Issues track

Table 1.1
Domain model entities

Entity	Role in the model
Applications	Business unit from asset inventories (Section 1.1): name, owning team, criticality tier, lifecycle status, compliance scope. Spans repositories and deployment environments.
Repositories	Technical asset from SCM (Section 1.2): primary unit around which findings are grouped.
Findings	Security issue from any tool category: severity, status, source tool, code location, rule identifier. Each traces to a specific scan.
Vulnerabilities	CVE identified issue enriched with three signals: CVSS from NVD, exploitation probability from EPSS, and CISA KEV confirmed exploitation. Multiple findings may reference the same vulnerability.
Teams	Organizational ownership linking personnel to applications and remediation responsibility.
Commits	Individual code changes with author, timestamp, files. Link findings to a code version.
Pull requests	Code change proposals where scans typically execute. Natural grouping unit for new findings.
Pipeline runs	CI/CD execution records: which tools ran, when, against which commit, gate outcomes.
Issues	Remediation work items in trackers such as Jira. Enable MTTR and SLA monitoring.
Dependencies	Third party libraries from SCA: package, version, license, CVEs.
Branch policies	SCM governance: required reviewers, status checks, merge restrictions. Indicate process maturity.

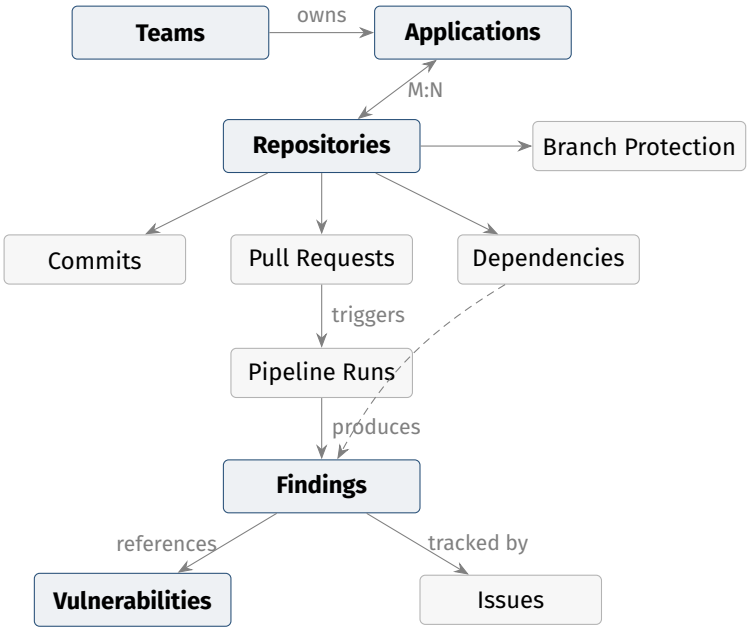

Figure 1.1
Conceptual Domain Model

remediation.

The structure maps to dimensional modeling (Kimball & Ross, 2013). Findings are facts. Applications, repositories, and teams are dimensions. Commits, pull requests, and pipeline runs add temporal and process dimensions. Cross tool deduplication collapses related findings while preserving traceability to each source.

Stakeholders consume this data through different patterns:

- Application owners need aggregated risk dashboards showing finding counts by severity, remediation progress, and compliance status.
- Developers need actionable findings with code level context, delivered through issue trackers.
- Security experts need drill down access to raw findings, audit trails, and cross tool correlation for triage.
- Leadership needs executive metrics: MTTR, SLA compliance, vulnerability density trends, and portfolio level comparisons.

These span the OLAP and OLTP requirements from Section 1.7.1 and drive the serving architecture in Chapter 2.

1.8 Related Work and Gap Analysis

No standard approach exists for consolidating security tool output into a unified data platform. Organizations face a fragmented set of partial solutions.

1.8.1 Existing Approaches

Gardner et al. (2023) define ASPM as tools that continuously manage application risk through collection, analysis, and prioritization of security issues across the software lifecycle. Commercial platforms such as Apiiro, Cycode, ArmorCode, and Snyk AppRisk aggregate findings into unified dashboards with correlation and risk scoring, but they are proprietary. Pipelines cannot be customized, custom ML cannot be added, and integrations are limited to what the vendor supports.

Platform native features cover only their own ecosystem. GitHub Advanced Security provides code scanning via CodeQL, secret scanning, and dependency review (GitHub, 2025a). GitLab bundles SAST, DAST, dependency scanning, container scanning, and a security dashboard (GitLab, 2025). Organizations using multiple SCM platforms or third party tools cannot consolidate through them.

DefectDojo (DefectDojo, 2024) is the most prominent open source alternative, supporting imports from over 150 tools through format specific parsers. It was designed as a vulnerability management interface, not a data platform. Its PostgreSQL backend limits enterprise scale analytics, and advanced API connectors are commercial only. OCSF (OCSF Project, 2024) standardizes security event schemas for SIEM and SOAR but does not model application level entities such as repositories, applications, or teams.

Many enterprises build ad hoc integrations: custom scripts, ETL pipelines, and dashboards from manual exports. These lack schema governance, break on API changes, and persist as institutional knowledge in unmaintained scripts.

1.8.2 Databricks Lakewatch

Databricks announced Lakewatch in March 2026 as an open, agentic SIEM built on the Databricks lakehouse (Databricks, 2026a). It unifies security, Information Technology (IT), and business data in a single governed environment powered by Delta Lake and Unity Catalog, with agentic triage, Detection as Code, and integrations from Wiz, Palo Alto Networks, Okta, and Zscaler.

Lakewatch focuses on runtime security operations: threat detection, incident response, alert triage, and log analytics. This framework focuses on application security posture: findings from SAST, SCA, DAST, secret scanning, and WAF runtime telemetry, joined to CMDB and SCM context for remediation tracking and risk aggregation. The two are complementary. Shared lakehouse infrastructure enables data exchange: Lakewatch could consume application risk scores for incident triage, and the framework could ingest Lakewatch's runtime detections as a finding source.

1.8.3 Comparative Analysis and Research Gap

Table 1.2 compares the surveyed approaches against criteria from the preceding analysis.

Table 1.2
Comparison of Existing Approaches

Criterion	ASPM	Platform native	DefectDojo	Lakewatch	This thesis
Open source	–	–	Yes	–	Yes
Vendor agnostic model	–	–	Partial	–	Partial
Extensible connectors	–	–	Yes	Partial	Yes
Lakehouse architecture	–	–	–	Yes	Yes
OLAP + OLTP serving	–	–	–	–	Yes
Data quality enforcement	–	–	–	–	Yes
AppSec specific model	Yes	Partial	Yes	–	Yes
Enterprise scalability	Yes	Partial	–	Yes	Partial

Note. Yes indicates full support, Partial indicates acknowledged but not demonstrated, – indicates out of scope or not addressed. Source: author synthesis of cited products' public documentation, accessed 2026-04.

No existing approach satisfies all criteria. Commercial ASPM is proprietary, platform native aggregations are ecosystem locked, DefectDojo lacks the data architecture for enterprise scale analytics, and Lakewatch addresses runtime operations rather than application security posture. The present work is itself partial on two criteria: the vendor agnostic claim holds at framework level but the reference implementation runs on Databricks alone, and lakehouse based enterprise scalability is supported by design but not empirically validated here (both noted in the Limitations section of the Conclusion). Academic work on application security consolidation as a data engineering problem is limited, focusing on detection techniques instead of cross tool integration at scale.

1.9 Selected Sources

The reference implementation in Chapter 3 instantiates the framework against nine source systems chosen to cover the ingestion and integration patterns that the framework must support, while keeping the sample small enough to manage. The selection spans all three detection tiers: static testing, dynamic testing, and runtime security.

1.9.1 Selection Criteria

The nine sources are selected based on five criteria:

1. **Open source or free tier available.** The reference implementation must be reproducible without commercial licenses.
2. **Full domain coverage across three detection tiers.** The nine together must cover every domain in the framework: CMDB, SCM, and the three detection tiers, namely static testing (SAST, SCA, secrets), dynamic testing (DAST), and runtime security (WAF).
3. **API heterogeneity.** The selection deliberately includes REST (ServiceNow, SonarQube, Dependency-Track, GitHub, GitLab, OWASP ZAP, AWS WAF), GraphQL (GitHub, GitLab), and CLI wrapped (Semgrep, TruffleHog) sources so the connector contract is exercised across integration styles.
4. **Limit heterogeneity to nine to maintain scope control.** The nine sources cover the ingestion and integration patterns that the framework must support. A wider sample would dilute pattern coverage without strengthening the framework claim.
5. **Industry adoption.** Each tool is in widespread enterprise use.

A summary of the variation in incremental strategies among the selected sources is presented in Table 1.3.

Table 1.3
Pairings of source and incremental strategy

Source	Incremental strategy
ServiceNow	Native `sys_updated_on` high water mark column
SonarQube, Dependency-Track	Native high water mark column (SonarQube `updateDate`, Dependency-Track `lastOccurrence`)
GitHub, GitLab	Webhook
Semgrep, TruffleHog	Commit SHA
OWASP ZAP	Scan lifecycle retrieval
AWS WAF	Firehose to S3 log stream (preferred), time window sampled retrieval (fallback)

Note. Classification synthesized from the medallion pattern treatment in Strengholt (2025) and the Lakeflow Declarative Pipelines operations guidance in (Databricks, 2025c). Pairings of source and strategy are author attributions based on the API capability matrix per source.

1.9.2 Source Inventory

The selection resolves to nine sources spanning the three detection tiers, plus a CMDB for inventory and an SCM pair for the framework first dependency.

- **ServiceNow** (CMDB). Dominant enterprise CMDB with REST Table and CMDB Instance APIs and a native `sys_updated_on` high water mark.
- **GitHub** (SCM plus integrated SAST, SCA, secrets). REST and GraphQL APIs with cursor pagination and rich webhooks.
- **GitLab** (SCM plus integrated security). REST and GraphQL with keyset and offset pagination. Security findings require Ultimate tier or pipeline artifacts.
- **SonarQube** (SAST). Mature server product with a REST Web API, `updateDate` high water mark, and a scan completion webhook.
- **Semgrep** (SAST). Free CLI engine deployed in a CI/CD step. Exercises the artifact ingestion pattern.
- **Dependency-Track** (SCA). OWASP project with a REST API and SBOM centric vulnerability correlation across multiple advisory sources.
- **TruffleHog** (secrets). CLI tool with live credential verification that populates the `validity_status` field.
- **OWASP ZAP** (DAST). Open source dynamic scanner operated as a daemon with a REST API (OWASP Foundation, 2024d), exercising the on demand scan lifecycle pattern.
- **AWS WAF** (WAF, runtime). Kinesis Data Firehose to S3 log stream as the primary path (Amazon Web Services, 2026), with the `GetSampledRequests` API as the time window sampled fallback.

Details for each source are on the connectors reference hub.

1.9.3 Considered and Excluded

Table 1.4 enumerates the key tools that were considered and the criterion that excluded each. Tools marked **within scope, deferred** satisfy all criteria but are excluded based on the cap in Criterion 4. Onboarding them would simply be a repeat of the procedure per source in Section 3.1.

1.9.4 Cross Source Synthesis

Across the nine sources, four record forms recur (ticket like, resource like, finding like, event like), severity scales span three to six levels, and status vocabularies diverge more sharply with tool specific states that rarely map one to one. Five pagination strategies appear: offset (ServiceNow, SonarQube, Dependency-Track), cursor (GitHub REST), GraphQL cursor (GitHub and GitLab GraphQL), keyset (GitLab REST), and none (CLI sources, AWS WAF). The sources also split on an operational pattern axis: periodic global scanners polled via `updated_at` (SonarQube, Dependency-Track), CI/CD step scanners indexed by commit SHA (Semgrep, TruffleHog), on demand dynamic scanners read via scan lifecycle API (OWASP ZAP), and runtime telemetry sampled over time windows (AWS WAF). Together they exercise all five

Table 1.4

Considered and excluded sources. Each row names the tool, the category it would have populated, and the criterion that excluded it (C1 open source or free tier, C2 full domain coverage, C3 API heterogeneity, C4 heterogeneity cap, C5 industry adoption).

Tool	Category	Excl. by	Note
Checkmarx	SAST	C1	commercial only, no permissive tier
Fortify	SAST	C1	commercial, OpenText
Burp Enterprise	DAST	C1	commercial. Community edition is manual only
Snyk	SCA / SAST	C4	**within scope, deferred** (overlap with Dependency-Track and Semgrep)
GitGuardian	secrets	C4	**within scope, deferred** (overlap with TruffleHog)
GitLeaks	secrets	C4	**within scope, deferred** (overlap with TruffleHog)
Dependabot	SCA	C4	**within scope, deferred**. GitHub native, overlap with Dependency-Track
Jira	issue tracker	C4	**within scope, deferred**. Issue tracking is in the data model (Section 1.7.3), MVP defers connector to keep the count at nine
Azure DevOps	SCM / CI/CD	C4	**within scope, deferred** (overlap with GitHub and GitLab)
Bitbucket Cloud	SCM	C4	**within scope, deferred**
Jenkins	CI/CD	C2	CI orchestrator, does not produce ASPM findings on its own
GitHub Actions	CI/CD	C4	**within scope, deferred**
Cloudflare WAF	WAF	C4	**within scope, deferred** (overlap with AWS WAF)
Akamai WAF	WAF	C1	commercial, no free tier

ingestion strategies the framework prescribes. The source capability matrix tabulates them against the capabilities that matter for connector design.

This thesis fills the gap with a vendor agnostic framework. It applies lakehouse architecture and the medallion pattern to ingest tool output, normalize it into a domain model, and serve it through OLAP and OLTP interfaces. Chapter 2 presents the framework, and Chapter 3 demonstrates its implementation.

2. Framework

Chapter 1 analyzed the sources, patterns, and gaps in enterprise application security data integration. This chapter presents the proposed framework: a reusable blueprint defining the architecture, data model, and patterns an organization follows to build its own implementation. The framework targets the Databricks Lakehouse, separating general patterns from platform specifics. Chapter 3 then demonstrates a concrete implementation.

2.1 Solution Architecture

This section presents the framework architecture at three levels: technology stack, component design, and data layer. The architecture addresses the requirements from domain analysis: heterogeneous source ingestion, vendor agnostic normalization, dual OLAP/OLTP data serving, and extensibility for new data sources and analytics.

A further design principle applies to the framework artifacts: every template defined in this chapter (schema patterns, mappings, connector contract) is declarative, not procedural, so that an AI coding agent reading the specification has enough material to produce a candidate implementation. The published skill catalog, which sits on the external specification site and wraps these templates into executable prompts, is evaluated separately in Section 3.6.3.

2.1.1 Technology Stack

The framework targets the Databricks platform on AWS. Databricks separates storage from compute: data resides in object storage and compute scales independently. This reduces costs for uneven workloads typical for security data integration (Armbrust et al., 2021).

The four platform components are Delta Lake, Unity Catalog, Lakeflow Declarative Pipelines, and Lakebase. **Delta Lake** (Armbrust et al., 2020; Lee et al., 2024) is the open table format providing the storage layer for all three medallion tiers, with ACID transactions, schema enforcement, schema evolution, and time travel. **Unity Catalog** (Databricks, 2025e) manages a three level namespace (`catalog.schema.table`) mapped to the medallion layout (one catalog per environment, schemas per source for bronze, per domain for silver and gold), with fine grained access control and automated lineage. **LDP** (Databricks, 2025c) declares pipelines as Python or Structured Query Language (SQL) functions, embeds data quality expectations as declarative constraints with quarantine on violation, and processes incrementally through change data feed. **Lakebase** (Databricks, 2026b) is a serverless PostgreSQL database that shares storage with the lakehouse and provides the OLTP serving layer required by Section 1.7.1.

Running all four on one platform yields governance, lineage, and compute benefits that would require significant integration across separate tools. The platform also lines up with Lakewatch, the Databricks SIEM analyzed in Section 1.8.2: both share Delta Lake storage and Unity Catalog governance, opening a path for the framework gold outputs (risk scores, remediation status) to enrich Lakewatch threat detection and for Lakewatch runtime events to feed back as a finding source.

2.1.2 Component Design

Five tiers organize the framework, illustrated in Figure 2.1. Data flows from top to bottom, from sources through the platform to consumers.

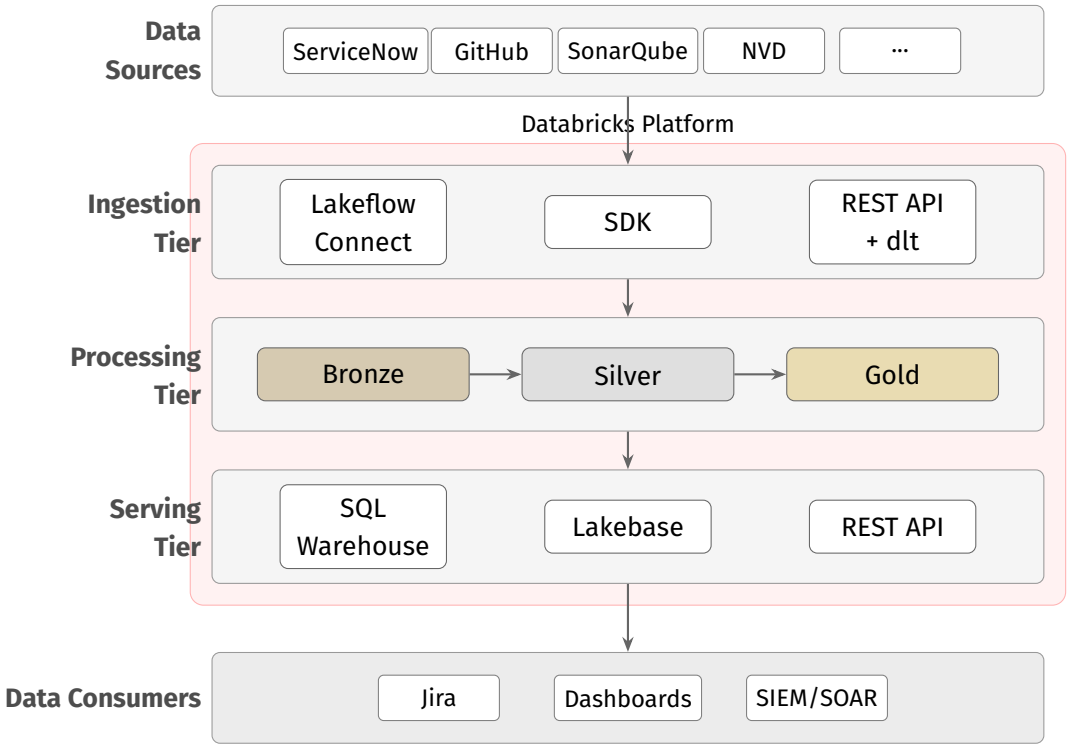

Figure 2.1
Component Design

Data sources are the external systems analyzed in Chapter 1: application inventories, SCM platforms, CI/CD tools, security scanners, and vulnerability databases. They are outside the framework boundary. The framework consumes their APIs but does not control them.

Ingestion tier contains connectors that extract data from sources and store it in the bronze layer. Three connector categories serve different integration scenarios. Lakeflow Connect uses a declarative managed ingestion pattern. SDK connectors use source provided client libraries for programmatic extraction. REST API connectors use the open source dlt library for sources which lack Lakeflow Connect or dedicated SDK. A fourth carve out, the artifact path, applies to CLI only tools that emit a report file during a continuous integration step

and is documented as a separate ingestion pattern at Section 2.4.2. All connectors share common concerns: authentication, pagination, rate limiting, and incremental state. The connector framework in Section 2.4 defines these patterns.

Processing tier implements the medallion architecture (Section 2.1.3). Delta Lake provides the storage layer. LDP orchestrates the transformations. Bronze stores raw ingested data in source native schemas. Silver normalizes it into the vendor agnostic entity and finding model defined in Section 1.7.3. Gold computes aggregated metrics, ML enriched scores, and consumption ready datasets. Unity Catalog governs access and tracks lineage across all three layers.

Serving tier exposes processed data to downstream consumers. SQL warehouses serve OLAP queries for dashboards and analytics. Lakebase serves OLTP workloads: low latency lookups and operational APIs. A REST API layer provides HTTP access to the OLTP store.

Data consumers are external systems reading from the serving tier: issue trackers (Jira, ServiceNow), SIEM/SOAR platforms, and custom reporting tools. Like data sources, consumers are outside the framework boundary.

2.1.3 Medallion Architecture

The processing tier applies the medallion architecture pattern from Section 1.7.1. Each layer is a Delta Lake schema within a single Unity Catalog, providing unified governance and cross layer lineage. Figure 2.2 illustrates the layer structure and transformations between them.

2.1.3.1 Bronze Layer

Bronze stores raw data from each source with no business logic, in a per source schema (`bronze_<source>`). Records are appended with the bronze envelope columns. Schema on read accepts new source fields without pipeline changes. Tables are partitioned by ingestion date. Structural validation failures at ingestion go to per source quarantine tables.

2.1.3.2 Silver Layer

Silver is the system of record. Transformations normalize source data into the vendor agnostic domain model from Section 1.7.3, applying severity harmonization, entity normalization, timestamp standardization, and deduplication. The layer holds entity tables (dimensions), the single `silver.findings` fact table (Section 2.2.3.2), and relationship tables

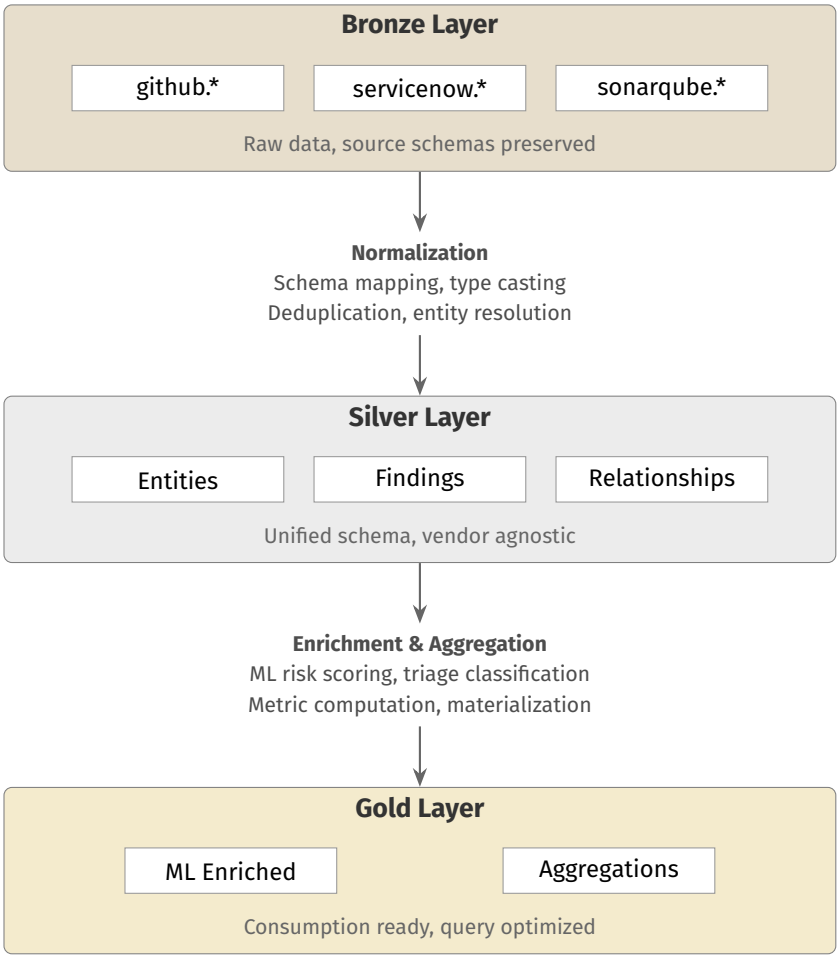

Figure 2.2
Medallion Layer Design

for many to many links. LDP expectations enforce constraints declaratively, and violators go to quarantine.

2.1.3.3 Gold Layer

Gold computes consumption ready datasets from silver. Aggregation tables compute metrics (risk scores, team remediation rates, MTTR, SLA compliance, time series). ML enriched tables store model outputs (composite risk scores, false positive predictions, remediation time estimates) per the patterns in Section 2.5.1. Gold uses incremental refresh where possible, and full refresh applies for metrics needing global recomputation.

2.2 Data Model

This section maps the conceptual domain model from Section 1.7.3 to physical table designs across the three medallion layers. Reusable schema patterns are applied to produce the concrete entity, finding, and aggregation tables that the framework provides.

2.2.1 Source Synthesis

The schema patterns are derived from the APIs of the nine sources introduced in Section 1.9, not chosen in advance. The per category capability matrix consolidates the observations relevant to schema design at the category level. The connectors reference hub provides facts for each source. This section maps the consolidated capabilities onto concrete Entity, Finding, and Relationship schemas.

2.2.2 Schema Patterns

Schema patterns standardize how tables are created at each medallion layer. Figure 2.3 walks a single SonarQube SAST finding through the layers and shows the metadata envelope from Section 2.2.2.1.

2.2.2.1 Bronze envelope

Every bronze table carries a uniform metadata envelope: four columns shared across connectors (`_ingestion_timestamp`, `_source_system`, `_batch_id`, `_raw_payload`) and a fifth (`_hwm_value`) on incremental only tables (Section 2.4.2). When the framework owns the bronze schema

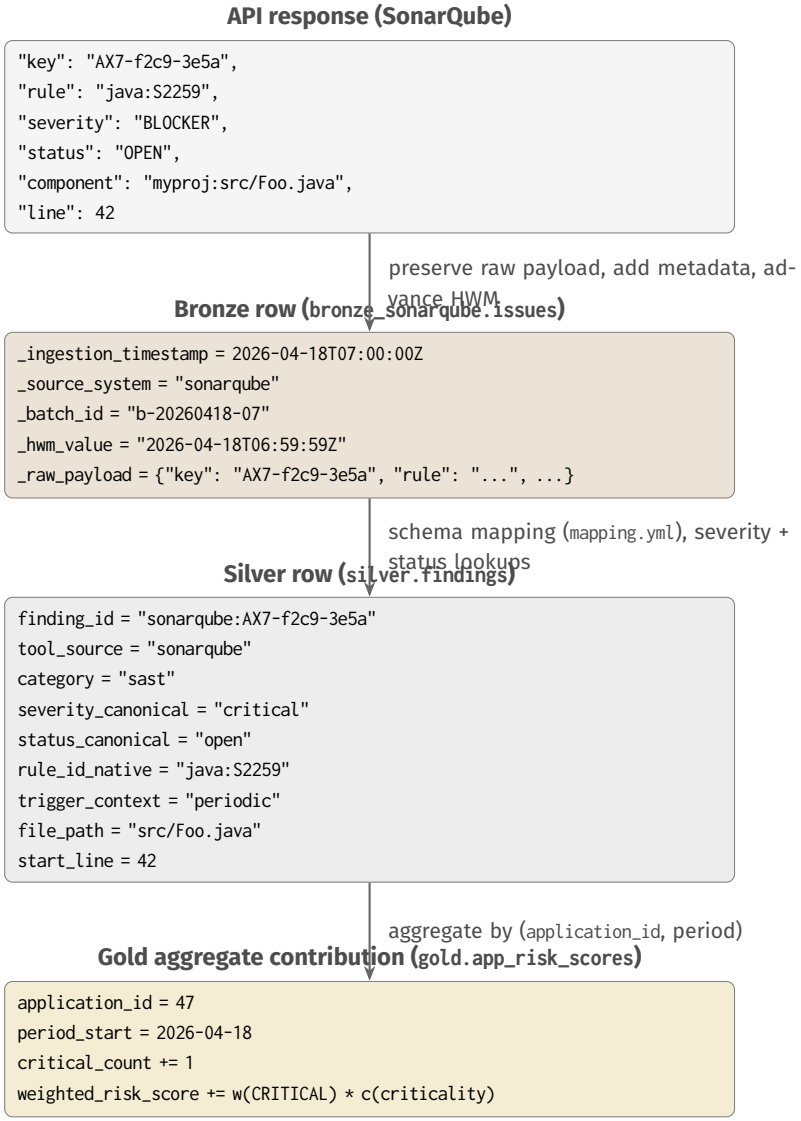

Figure 2.3

Lifecycle of a single SonarQube SAST finding from API response through Bronze ingestion with the metadata envelope with five columns, Silver normalization into `silver.findings` *with category discriminator, and Gold aggregation into* `app_risk_scores`*.*

(artifact sources such as OWASP ZAP and Semgrep), a helper stamps the envelope before the write. When an ingestion managed service owns the table (Lakeflow Connect, as in ServiceNow), a downstream SQL view projects the same five columns on top of the managed table using the service run identifier and timestamp. Downstream readers see the same envelope regardless of ingestion path.

Additional source native columns sit alongside the raw payload via schema on read. New fields are accepted through additive evolution. Tables are partitioned by ingestion date.

2.2.2.2 Silver entity pattern

Entity tables store normalized dimension data. The natural key plus source system identifies a record origin, and the surrogate key provides a stable foreign key target even when source identifiers change (Kimball & Ross, 2013). The framework specifies the Slowly Changing Dimension (SCD)-2 pattern (`valid_from`, `valid_to`) for tracking entity history (Inmon, 2005; Kimball & Ross, 2013). The reference implementation in Chapter 3 simplifies to SCD-1 (overwrite on update) since the MVP scenarios do not exercise point in time queries. The field derivation table per source is published at https://vkraus.github.io/appsec-mvp/platform/reference/canonical-mapping/#silver-entity-mapping-requirements.

The Bronze to Silver step quarantines a record when (1) the Bronze record failed JSON parse at landing, (2) a required field is null (`natural_key`, `source_system`, `valid_from`), or (3) a mandatory lookup yields no match. Optional field nulls and safe casts are accepted with a data quality warning. The same rule applies to finding tables.

For CMDB entities, related tables (`cmdb_rel_ci`, group and user references) are ingested as separate Bronze tables and joined in Silver rather than resolved through relationship APIs. This keeps `ingest.py` stateless and localizes schema change handling to the transformation layer.

2.2.2.3 Silver finding pattern

The single Silver Finding table `silver.findings` (rationale in Section 2.2.3.2) stores normalized fact data for every category. Each target field unions over the native fields of the finding emitting sources, with inapplicable fields stored as `NULL`. The `mapping.yml` per source (Section 2.4.3) makes each mapping explicit, including the `category` discriminator. Per source derivation tables are published at https://vkraus.github.io/appsec-mvp/platform/reference/canonical-mapping/#silver-finding-mapping-requirements.

2.2.2.4 Deduplication

When several tools scan the same artifact, overlapping findings are linked rather than collapsed (Reis & Housley, 2022). Deduplication is applied within `silver.findings` using a category conditional exact match tuple:

- SAST (`category = 'sast'`): `(repository_id, file_path, rule_id, line_number)`.
- SCA (`category = 'sca'`): `(repository_id, package_name, cve_id)`.
- Secret (`category = 'secret'`): `(repository_id, commit_sha, secret_type, file_path)`. Secrets are retained per commit, not collapsed by value.

A match across two tools links the records via a `dedup_links` record. No fuzzy matching is performed. When tools diverge on line numbers, both findings are retained and grouped at Gold via a `finding_group_id` on `(repository_id, file_path, rule_id)` independent of line number.

2.2.2.5 Relationship pattern

Relationship tables hold many to many mappings between entities: foreign keys to both sides, a source system indicator, and `valid_from`/`valid_to`. No payload columns. Examples are application to repository, finding to CVE, and cross tool deduplication links.

2.2.2.6 Gold aggregation

Aggregation tables follow a grain, metric, period structure (Kimball & Ross, 2013): grain columns define detail level (entity key plus time period), metric columns store computed values (finding counts, MTTR, SLA compliance, risk score), period columns store the window (`period_start`, `period_end`), and refresh metadata records the `computed_at` timestamp and strategy.

2.2.3 Entity Model

The silver layer instantiates the schema patterns as concrete tables in four groups: entities (dimensions), findings (facts), reference data, and relationships. Figure 2.4 shows a representative subset that exercises every relationship type. The prescription totals 15 tables across the four groups. The MVP in Chapter 3 realizes the four table subset {`applications`, `repositories`, `findings`, `app_repo_mapping`} sufficient to demonstrate the medallion contract end to end. Population of the remaining tables is recorded as Future Work in Section 3.6.3.

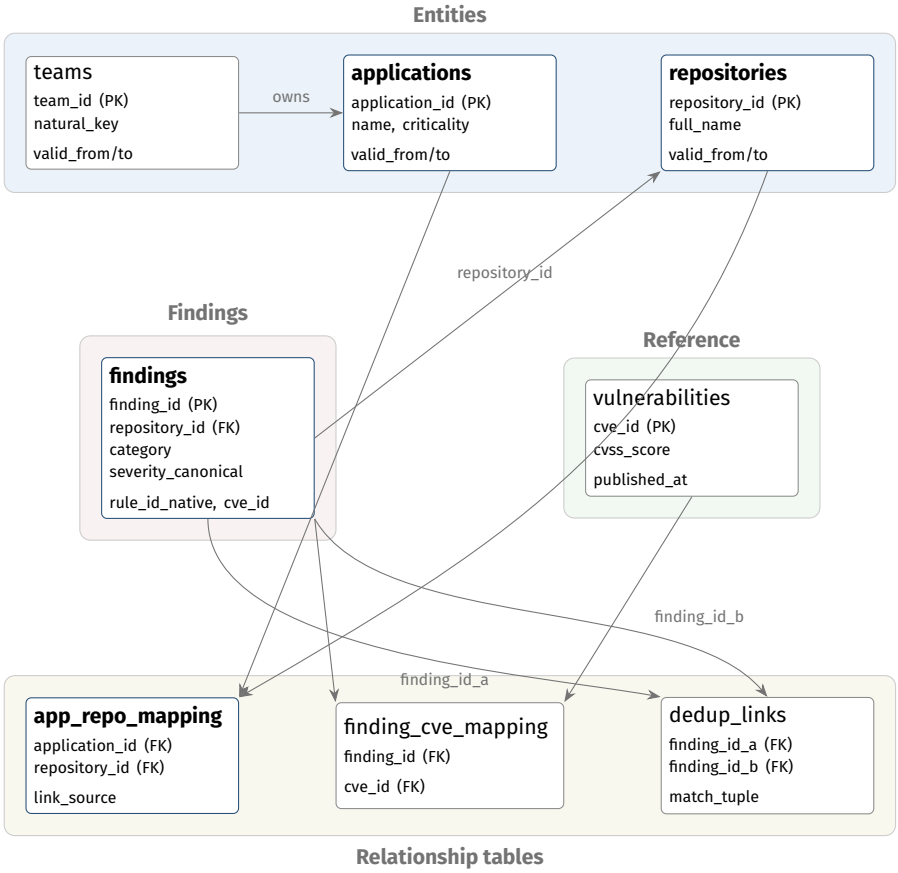

Figure 2.4

*Silver layer entity relationship diagram for the prescribed framework. Representative subset: **8** of the **15** prescribed silver tables grouped by type (entities, findings, reference, relationships). Highlighted nodes are the four tables realized by the MVP in Chapter 3, which simplifies SCD-2 to overwrite on update. The remaining tables follow the same schema patterns and are listed as Future Work.*

2.2.3.1 Entity Tables

Eight entity tables hold the stable business objects: `applications` and `repositories` as the two primary tables, plus `teams`, `commits`, `pull_requests`, `pipeline_runs`, `dependencies`, and `branch_policies`. All tables share the silver entity pattern columns (`id`, `natural_key`, `source_system`, `valid_from`, `valid_to`). Domain specific columns and source mappings are listed on the silver table ownership reference page.

2.2.3.2 Finding Table

All security findings land in a single Silver table, `silver.findings`. A `category` column discriminates records as `sast`, `sca`, `secret`, `dast`, `waf`, `container`, or `iac`. Category specific attributes (such as `file_path` and `cwe_id` for SAST, or `cve_id` and `installed_version` for SCA) are carried as nullable columns and populated only where the category defines them. Code located findings reference their repository through `repository_id`. Runtime perimeter findings (WAF) leave `repository_id` null and use `url` as the closest location coordinate. Business application context is derived through the `app_repo_mapping` relationship table instead of being stored on the finding itself.

A single table is preferred over per category tables because the silver mapping is already a union over sources (Kimball & Ross, 2013). Splitting it would force `UNION ALL` for cross category analytics and fragment the per category dedup routing. At the target scale, Delta Lake columnar compression makes the sparse NULLs cheap. Full column listing and rationale on the single table findings rationale page.

2.2.3.3 Reference and Relationship Tables

Three reference tables hold external intelligence: `vulnerabilities` (CVE records with CVSS from the NVD), `epss_scores` (daily EPSS probabilities per CVE), and `kev_entries` (CISA KEV catalog). They are refreshed on schedule and linked to findings through CVE identifiers, supporting the three signal enrichment described in Section 1.3.

Three relationship tables implement the cross object mappings: `app_repo_mapping` (applications to repositories, many to many), `finding_cve_mapping` (findings to CVE records), and `dedup_links` (duplicate findings across tools, with one reference record). Silver tables and connectors do not line up one to one. Each table can be fed by multiple connectors and each connector feeds multiple tables.

2.2.4 Aggregation Model

Four gold tables target the stakeholder needs identified in Section 1.7.3, each following the gold aggregation pattern from Section 2.2.2.

- `app_risk_scores`: composite risk per application per period. Open finding counts by severity, a weighted risk score combining severity, EPSS probability and application criticality tier, SLA compliance, and a trend indicator. Serves application owners.
- `team_metrics`: remediation performance per team per period. MTTR by severity, closure rate, new versus resolved ratio, SLA breach count. Used by leadership for cross team comparisons.
- `vulnerability_trends`: time series at configurable intervals (daily, weekly, monthly). New findings, resolved findings, net open count, severity distribution shifts, mean age of open findings. Powers longitudinal dashboards.
- `coverage_analysis`: gaps in tool coverage per repository. A repository with no SAST findings and no SAST pipeline runs likely lacks static analysis integration. Helps prioritize tooling rollout.

Adding a new gold table follows the same procedure: define the granularity, specify the metrics, write the LDP transformation, configure the refresh strategy. The shared aggregation pattern lets dashboards consume new tables without structural changes.

2.3 Environment and Deployment

Deployment assumes a working Databricks environment. Account onboarding, workspace creation, networking, and Identity and Access Management (IAM) federation are outside scope. Deployment splits into two tiers along resource ownership: every Databricks object that the bundle format has a native type for is declared in a Databricks Asset Bundle, and source side cloud infrastructure for each connector is declared in Terraform under the connector folder. The deployment model follows the declarative infrastructure as code patterns established in the DevOps literature (Forsgren et al., 2018; Kim et al., 2021).

2.3.1 Deployment Strategy

The **Databricks tier** owns every workspace resource the bundle format covers natively: catalogs, schemas, volumes, jobs, LDP pipelines, notebooks, permissions, cluster attachments, and the Lakeflow Connect connection objects. Databricks Asset Bundles (Databricks, 2025d) declare these alongside the source code, so one `databricks bundle deploy` recreates the workspace state for a new environment. The handful of Databricks objects the bundle format does not yet have a native type for (the secret scope container, Unity Catalog storage credential, and Unity Catalog external location) are created by a small post deploy shell

script colocated with the platform layer at `src/platform/scripts/bootstrap.sh`. The framework deliberately keeps a single state owner per resource. Mixing the bundle with the Databricks Terraform provider would split state ownership for the same Databricks objects across two state stores and create promotion drift.

The **source side tier** owns cloud infrastructure for each connector that needs to bring up its source system or supporting cloud resources (S3 buckets, IAM roles, OIDC trust policies, source side VMs). Each connector ships a Terraform runtime under `src/connectors/<source>/runtime/`, scoped to that connector and never referencing Databricks resources. This keeps the layering rule of Section 3.2.1 clean: per connector setup code has no upward or sideways dependencies. Organizations preferring Pulumi or OpenTofu can substitute those at this tier without touching the Databricks tier.

One bundle exists per target, with three targets structuring the promotion path (development, staging, production). Each target overrides the catalog name, secret scope, and compute target. Both tiers are declarative, so the same flow moves a change through the three environments without hand editing.

2.3.2 Project Structure

The project is a monorepo: `src/` for pipeline source code, `tests/` for the test suite, `config/` for lookup tables, and bundle files at the root. The Unity Catalog layout mirrors the medallion layers (Section 2.1.3) with one catalog per environment (`appsec_dev`, `appsec_staging`, `appsec_prod`). Source specific bronze and silver tables live in `bronze_<source>` and `silver_<source>` schemas. Cross source silver and gold tables live in unqualified `silver` and `gold` schemas whose names encode the analytic, not the source. Pipeline names mirror their source module (`ingest_<source>`, `transform_<source>_silver`). Column names use `snake_case` throughout. These conventions let governance rules target tables and pipelines by name pattern.

Each connector module under `src/connectors/{source}/` carries the same artifacts (Section 2.4.1): `ingest.py`, `transform.py`, `mapping.yml`, `config.yml`, and a colocated `tests/` folder. Silver to gold transformations group by analytic rather than by source because a single gold table can consume data from multiple connectors.

Secrets sit in the platform secret scope and are referenced by name. Nonsensitive settings (severity, status lookups) sit colocated with the connector code as YAML. Tuning these values is a configuration change, not a code change. Full layout with a worked example on the project layout reference page.

2.3.3 Pipeline Orchestration

Lakeflow Jobs schedules pipeline execution (Databricks, 2026c). A job groups tasks into a Directed Acyclic Graph (DAG) with explicit dependencies. Jobs are defined in the bundle alongside the resources they operate on, so scheduling promotes through the deployment path of Section 2.3.1.

One job per connector handles two sequential tasks: ingest into bronze, then transform to silver. This gives each connector an independent failure domain. Gold layer analytics run as separate jobs that list their upstream connector jobs as dependencies, so a gold job starts only after the required silver data is fresh.

Source characteristics drive frequency: high change sources (SCM, scanners) run hourly, stable sources (CMDB) run daily, enrichment sources (NVD, EPSS, CISA KEV) follow their own cadences. Each task has retry count and backoff interval. On retry exhaustion the task fails and downstream tasks do not execute. Jobs run in parallel on ephemeral clusters.

2.3.4 Monitoring and Observability

Monitoring handles whether pipelines are working over time. The platform provides system tables for job execution history, pipeline events, and compute utilization (Databricks, 2026d). The framework tracks three dimensions: **data freshness** (last successful ingestion timestamp against thresholds from the scheduling frequency in Section 2.3.3), **pipeline health** (success rate, mean run duration, retry frequency, quarantine volume, with a rising quarantine rate preceding data quality issues), and **data quality trends** (violations of LDP expectations over time). Alerts fire at job level (individual failures) and system level (aggregate degradation such as multiple connectors stale simultaneously). Thresholds and delivery channels are configuration, so operators tune sensitivity without redeploying pipelines.

2.3.5 Testing and Validation

Deployment level verification covers the environment and codebase. Component level patterns live in Section 2.4.4 and Section 2.5.3. The CI/CD pipeline enforces linting, formatting, and unit tests over isolated helpers (severity lookups, timestamp parsers, schema mapping). After deployment, smoke tests confirm Unity Catalog objects, compute resources, secret scopes, and LDP pipelines exist with the expected configuration.

Tests are linked to requirement identifiers via pytest markers (`@pytest.mark.requirement("REQ-TRF-SEV")`) drawn from the requirement catalog. A traceability matrix generated from test results is published at `https://vkraus.github.io/appsec-mvp/platform/reference/catalog/#per-source-traceability-matrix` and applies uniformly across environment, connector, and analytics testing.

2.4 Connector Framework

The connector framework defines how data moves from the sources analyzed in Chapter 1 into the medallion layers from Section 2.1.3. Section 2.1.2 introduced three connector categories (Lakeflow Connect, SDK, and REST API with dlt). This section specifies the common abstraction they share, the patterns for landing data in bronze, and the patterns for transforming it to silver. The goal is a repeatable recipe: adding a new source implements a well defined set of concerns against a new API, not a pipeline redesign.

Figure 2.5 gives the end to end view refined by the subsections below: ingest, transform, and aggregate tasks along the spine, quarantine branches where validation can fail, and the state and dedup annotations that the framework commits to.

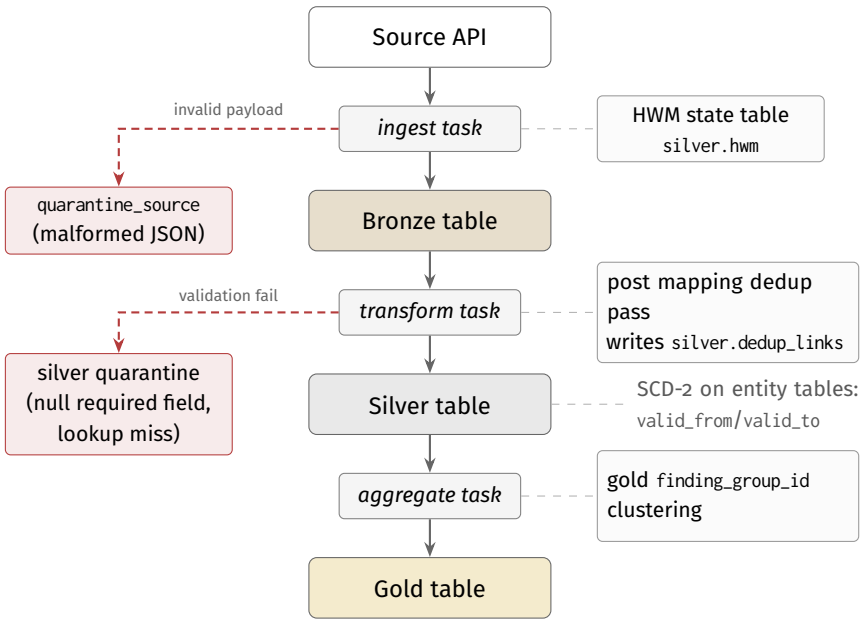

Figure 2.5
Data plane flow within the framework: ingestion with auditable high water mark state in `silver.hwm` *(right), quarantine on the Bronze side and on the Silver side for records that fail structural or required field validation (left), SCD-2 tracking on entity tables via* `valid_from/valid_to`*, a post mapping dedup pass that writes* `silver.dedup_links` *during the transform task, and gold* `finding_group_id` *clustering at the aggregate task.*

2.4.1 Connector Abstraction

Every connector handles the same cross cutting concerns, but the three categories from Section 2.1.2 differ in how much work the developer performs versus the platform or library.

2.4.1.1 Connector Categories

The three categories rank in a preference order set by how many of the five cross cutting concerns (auth, pagination, rate limit, incremental state, schema mapping) the developer must write versus delegate.

1. **Lakeflow Connect.** Declarative, configured through the bundle, not coded. Authentication, pagination, rate limiting, incremental state, and schema inference are platform handled. Suits sources with a supported Lakeflow Connect integration where full table ingestion meets requirements. The connector becomes a declaration that cannot drift from the framework contract.
2. **SDK connectors.** Use a source provided client library (AWS SDK for Python, PyGitHub, python-gitlab) that encapsulates transport, pagination, and rate limiting. The developer controls extraction logic. The library is typed against the source object model, so upstream API changes appear as library upgrades, not silent breakage in hand written code.
3. **REST API with dlt.** The open source dlt library provides prebuilt authentication, pagination, and incremental loading components composed declaratively. The developer specifies endpoints, schema mapping, and extraction parameters. More connector code than the prior two categories, but mechanical concerns still sit in tested library components rather than hand written HTTP.

Each step shifts more concerns from platform custody into connector code. Hand written HTTP clients sit outside the order entirely and are not sanctioned: they reintroduce every concern the three categories above delegate. Chapter 3 demonstrates all three categories. Figure 2.6 summarizes the selection.

2.4.1.2 Cross cutting concerns

2.4.1.2.1 Authentication

Source APIs use PATs, OAuth 2.0 client credentials, or service account keys. Credentials live in the platform secret scope. Each connector declares which type it requires and the framework resolves it at runtime. This keeps secrets out of code and enables credential rotation without redeploying.

2.4.1.2.2 Pagination

REST APIs return results in pages. Two strategies cover most sources: offset based (page number and size) and cursor based (opaque token). The connector abstraction hides

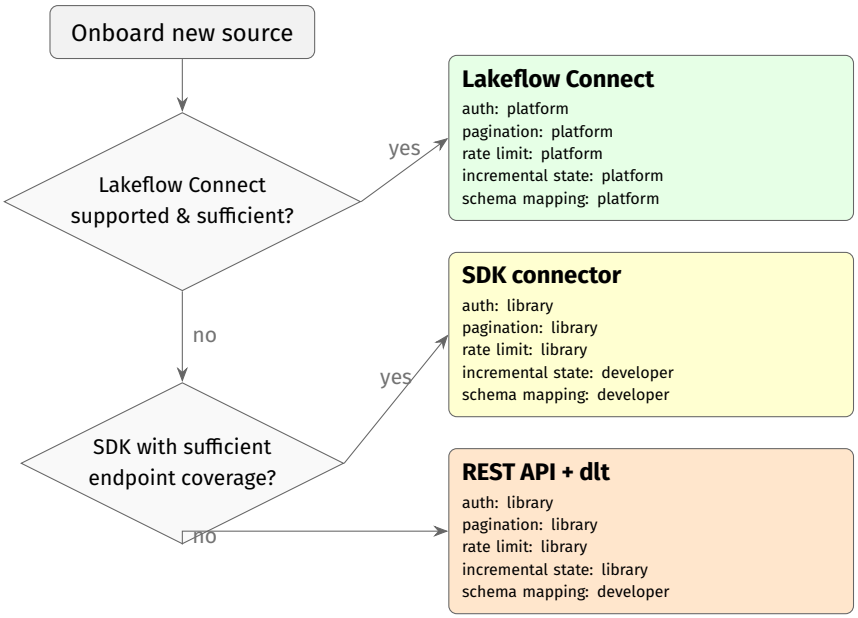

Figure 2.6
*Connector category selection. Terminal panels show how the five cross cutting concerns are handled: **platform** (managed), **library** (SDK or dlt), or **developer** (code per connector).*

pagination so downstream code sees a stream of records. GraphQL sources (GitHub API) use cursor based pagination exclusively.

2.4.1.2.3 Rate limiting

On HTTP 429 the connector pauses for the duration in the response headers before retrying. For sources without rate limit headers, a configurable per source rate limit prevents exceeding undocumented thresholds. Transient errors (HTTP 5xx) trigger exponential backoff with a configurable retry cap.

2.4.1.2.4 Incremental state

Full re-ingestion does not scale at enterprise volumes. Each connector maintains a high water mark (timestamp or cursor) persisted in a state table within the bronze schema. For sources supporting CDC, the connector consumes change events directly. Two patterns drive the choice. **Periodic global** sources (server scanners, CMDB, SCM polling `updated_at`) use a timestamp high water mark. **CI/CD step** sources (per commit CLI tools) use a per repository commit SHA or run identifier. Both patterns share the same state table schema.

2.4.1.3 Connector Contract

Every connector module exposes two entry points. `ingest` accepts a run identifier and state object, pulls new records, and returns a batch descriptor with the rows landed and

the advanced high water mark. `transform` accepts a bronze dataframe and returns a silver dataframe conforming to the entity or finding pattern (Section 2.2.2). Neither writes to silver itself. The pipeline runner persists the returned dataframe, so both entry points are testable with in memory fixtures. For artifact path connectors whose source specific ingestion primitive already uses the `ingest` symbol, the contract wrapper is exposed as `ingest_contract`.

The state object has a fixed structure across categories: high water mark value, source system identifier, and optional backfill parameters. The runner loads state before `ingest` and persists it after a successful write, giving consistent resume semantics regardless of category. Lakeflow Connect connectors satisfy the contract through platform configuration. SDK and REST API connectors implement the two operations directly.

2.4.1.4 Connector Artifacts

Adding a connector populates the module template from Section 2.3.2 with the same set of artifacts across all three categories:

1. `config.yml`: source parameters (base URL, endpoints, pagination strategy, rate limit, high water mark column, target bronze table).
2. A secret scope entry registering the connector credential, referenced from `config.yml`.
3. `ingest.py`: implements `ingest(run_id, state) -> batch`. Lakeflow Connect connectors leave this empty and declare the resource in the bundle fragment. SDK connectors delegate to the client library, and dlt connectors compose prebuilt components.
4. `transform.py`: implements `transform(bronze_df) -> silver_df` targeting the silver table for the category (Section 2.2.3).
5. `mapping.yml`, `severity.yml`, `status.yml`: declarative column expressions and lookups, maintained as configuration so vocabulary updates do not require a pipeline redeploy.
6. A bundle fragment under `resources/` declaring the two task connector job (Section 2.4.2).
7. `tests/`: ingestion and transformation tests per Section 2.4.4.

Silver and gold do not change: the silver tables from Section 2.2.3 take the new source through the schema mapping and gold analytics keep working.

2.4.2 Ingestion Patterns

Ingestion moves data from source APIs into bronze. All connectors follow the same landing pattern, with source type specific variations below.

2.4.2.1 Common landing pattern

Every ingestion run produces append only writes to the target bronze table. The full API response is preserved in `_raw_payload` alongside the bronze envelope columns (Section 2.2.2). No field filtering or transformation occurs. Bronze is an exact copy. The batch identifier enables idempotent replays: a failed run reexecutes the same batch and overwrites only those records. For sources where merge semantics are appropriate (e.g., CMDB entity snapshots), the connector upserts on the natural identifier. Records failing structural validation at landing (malformed JSON, unexpected schema changes) are routed to quarantine tables per source with raw payload, error, and batch identifier. No record is silently dropped (DAMA International, 2017; Reis & Housley, 2022).

2.4.2.2 Bronze table template

Every bronze table follows the same structure extending the schema pattern from Section 2.2.2. The `_hwm_value` column carries the high water mark per record, making the resume position auditable and supporting targeted rebuilds. Tables are partitioned by ingestion date. The naming convention is `appsec_<env>.bronze_<source>.<entity>` (for example, `appsec_prod.bronze_github.code_scanning_alerts`). A connector author fills in three blanks per entity: source name, entity name, and high water mark source field.

2.4.2.3 Connector job template

Every connector instantiates the same Lakeflow Job layout (Section 2.3.3): a two task DAG where ingest produces bronze and transform consumes it to produce silver, with the transform task declaring a hard dependency on the ingest task. Retry configuration is identical across connectors (three attempts, capped exponential backoff). The bundle fragment exposes a fixed set of parameters (source name, target catalog, high water mark reset flag, cron expression). Full fragment on the connector job template reference page.

The common landing pattern and bronze table template apply uniformly across categories. Per source parameters live in `src/connectors/{source}/config.yml`. Category specific notes (incremental strategy preference for SCM, scanner deployment styles, CMDB relationship traversal) are on the per category capability matrix.

2.4.3 Transformation Patterns

Transformations move data from bronze to the silver entity and finding tables from Section 2.2.3. Each transformation is a LDP pipeline reading from one or more bronze tables and writing to a silver table. Five patterns compose the path from bronze to silver.

2.4.3.1 Schema mapping

Each connector has a schema mapping that extracts typed columns from the bronze raw payload, applies type casts, and assigns the surrogate key. Mappings are declared as column expressions in the LDP pipeline. The `mapping.yml` for every connector has the same structure: a target table declaration and column blocks naming the target column, the JSON path into `_raw_payload`, and the cast. Severity and status lookups live separately as key value maps. A worked example (SonarQube) is at https://vkraus.github.io/appsec-mvp/connectors/sast/sonarqube/#mapping-example.

2.4.3.2 Normalization

Three rules bring source data to a common form:

- **Severity harmonization.** Tools use different scales (catalogued in Section 1.9). The framework maps each native scale to a four level model (critical, high, medium, low) through per tool lookup tables. Undocumented or missing values fall through to a configurable default (`medium`) and trigger a data quality warning.
- **Status normalization.** Lifecycle states vary across tools. A per tool status mapping translates them to a five state model (open, confirmed, resolved, false_positive, wontfix).
- **Timestamp standardization.** All timestamps are parsed during schema mapping and stored as Coordinated Universal Time (UTC) datetime columns.

2.4.3.3 Data quality validation

LDP expectations enforce constraints on every record entering silver. Expectations are declared alongside the transformation and checked at runtime. Examples: severity must be one of four standard values, status must be one of five standard states, repository foreign keys must reference an existing silver entity, and timestamps must fall within a plausible range. Violating records are quarantined, not propagated, consistent with the quarantine pattern at ingestion.

2.4.3.4 Deduplication application

Deduplication keys are defined in the Silver Finding pattern (Section 2.2.2). The transformation runs a post mapping dedup pass that writes `dedup_links` records for every overlapping pair sharing a `repository_id`. Deduplicated findings are not deleted. The link preserves traceability while establishing the reference finding. No fuzzy matching is performed, and line number divergence is handled at Gold via `finding_group_id`.

2.4.3.5 Business context attribution

Findings are linked to business applications through the `app_repo_mapping` relationship table. Because the mapping is many to many, findings inherit application context through a join instead of a direct foreign key, which enables the per application aggregations in Section 2.2.4. The mapping is sourced from the CMDB connector and can be supplemented by automated inference in the ML workflows from Section 2.5.1.

2.4.4 Testing and Validation

Connector tests live beside the connector at `src/connectors/{source}/tests/`, not in a central suite. `test_ingest.py` covers the ingestion contract, `test_transform.py` covers the transformation, and `fixtures/` holds JSON inputs named `{endpoint}_{scenario}.json`. Fixtures are recorded once from a representative source instance and frozen, so the suite runs in CI/CD without network access. The stack adds three libraries on top of pytest: an HTTP mocking library, a PySpark DataFrame assertion library (`chispa`), and LDP expectations for runtime checks. Each test carries a `@pytest.mark.requirement(...)` marker so the traceability matrix links results to the requirements below.

- `REQ-ING-AUTH`: authentication and credential resolution against the platform secret scope.
- `REQ-ING-PAG`: pagination traversal without data loss or duplication across at least two pages.
- `REQ-ING-RL`: HTTP 429 handling and backoff according to the configured retry policy.
- `REQ-ING-HWM`: high water mark resume across two consecutive runs with a midrun data change.
- `REQ-TRF-MAP`: schema mapping for every ingested endpoint.
- `REQ-TRF-SEV`: severity normalization covering every source specific severity value, including edge cases.
- `REQ-TRF-STS`: status normalization covering every source specific lifecycle state.
- `REQ-TRF-TS`: timestamp normalization for every source specific format.
- `REQ-DQ`: at least one LDP expectation per target silver table, with one violating and one valid record.
- `REQ-DEDUP`: deduplication for every applicable tool overlap pair (omitted when the connector category does not participate in dedup).

New connectors do not merge until every applicable marker above passes in CI/CD.

2.5 Analytics and Serving Framework

This section addresses the final stage: computing consumption ready insights from silver data and delivering them to stakeholders. Section 2.2.4 defined **which** gold tables the framework provides. This section defines **how** those computations are structured and how results reach consumers through analytical, operational, and event driven channels.

2.5.1 Analytics Patterns

Gold layer computations fall into two categories: rule based analytics applying deterministic formulas to silver data, and ML driven analytics learning patterns from historical data. Both produce outputs following the gold aggregation pattern from Section 2.2.2 and run under the *aggregate task* of the data plane (Figure 2.5).

2.5.1.1 Extension blueprint

Adding an analytics workflow follows the same procedure as a connector: define the business question and stakeholder, choose rule based or ML, implement as a LDP pipeline or MLflow experiment, then wire the output to a gold table with the configured refresh strategy. The artifacts under `src/analytics/{name}/` mirror the connector layout: `pipeline.py`, `ddl.sql`, `config.yml`, a bundle fragment under `resources/`, and `tests/`. Silver is the stable boundary: authoring a new analytic never requires connector edits, and adding a connector never requires analytics edits.

2.5.1.2 Rule Based Analytics

Three patterns cover the rule based work. **Composite scoring** combines signals into a single metric (the application risk score weights open findings by severity, multiplies by EPSS, and factors in criticality tier, with configurable weights). **Time series aggregation** rolls up findings at daily, weekly, and monthly intervals with period metrics and derived indicators (period over period change, moving averages). **Threshold classification** assigns categorical labels (risk tiers, coverage gaps, SLA breaches) downstream consumers can filter and alert on. Refresh is incremental where each record maps to a single grain partition. Full refresh applies for metrics needing global state (cross application percentiles, organization wide distributions).

2.5.1.3 ML Driven Analytics

ML analytics learn from historical silver and gold data. The framework supports four use cases: **composite risk scoring** learns which signal combinations preceded security incidents, **false positive prediction** estimates probability from historical triage decisions, **remediation time estimation** predicts MTTR for SLA forecasting, and **anomaly detection** flags spikes in finding volume or severity shifts. The exploit prediction direction draws on the published EPSS methodology (Jacobs et al., 2020).

Framework commitments across the four are narrow. Outputs land in Gold tables under the aggregation pattern, every run is tracked in MLflow with the run ID recorded on output rows, and retraining ships in the same Databricks Asset Bundle (DAB) bundle as ingestion so deploys and rollbacks are atomic. Feature Store supplies reusable feature tables, and Model Serving caches real time inference results in Lakebase (Databricks, 2025a; MLflow, 2025). Feature selection, splits, algorithms, and cadence are left to Chapter 3.

2.5.2 Serving Patterns

The serving layer delivers gold outputs to the data consumers from Section 2.1.2. Three delivery modes address different access patterns and latency requirements.

2.5.2.1 Analytical serving

Analytical serving targets dashboards and ad hoc analysis through the OLAP path. A SQL warehouse queries gold Delta tables, and materialized views cache recurring query patterns. Stakeholder specific SQL views (executive overviews, team dashboards, security engineering views) sit on top of the gold tables rather than as separate copies, keeping consumers consistent.

2.5.2.2 Operational serving

Operational serving targets low latency workloads through the OLTP path. Lakebase exposes gold tables as PostgreSQL queryable relations with sub 50 millisecond response times (Databricks, 2026b), sharing storage with the lakehouse so no synchronization pipelines are needed. Three workloads use this path: remediation state lookups for developer workflow integration, operational APIs exposed through PostgREST without custom API code, and cached ML inference results for real time risk assessment.

2.5.2.3 Event driven serving

Event driven serving pushes data to external systems instead of waiting for queries.

Automated issue creation triggers on new critical findings or SLA breaches, creating tickets in issue trackers (Jira, ServiceNow) with idempotency guards against reruns.

Threshold based notifications alert stakeholders when metrics cross configured boundaries (risk score, new KEV listing, SLA drop) through configured channels.

SIEM and SOAR feeds push events to platforms such as Lakewatch (Section 1.8.2) via CDC on gold tables. Shared infrastructure with Lakewatch removes the need for external data movement.

Bidirectional synchronization flows external status updates back to silver via scheduled polling or webhooks, so gold remediation metrics reflect actual resolution progress.

2.5.3 Testing and Validation

Rule based analytics are validated by snapshot comparison. Each test supplies silver inputs with known values and confirms the gold output matches expected exactly via PySpark DataFrame assertions. Test cases cover computation boundaries (zero findings, all critical, missing enrichment, period bucketing at month boundaries, threshold tier cutoffs) plus a refresh idempotency case that runs the pipeline twice on the same input.

ML analytics are validated by held out evaluation and drift monitoring. MLflow's `evaluate()` computes accuracy, precision, recall, and F1, and `validate_evaluation_results()` gates promotion against configured thresholds and the registered baseline (MLflow, 2025). In production, prediction distribution comparisons flag drift before model quality degrades. Serving endpoints are exercised through HTTP clients to verify correctness and latency against the targets in Section 2.5.2.

The suite covers every gold table (metric correctness, boundaries, idempotency), every ML model (convergence, accuracy, drift, promotion gating), and every serving endpoint (correctness and latency). New analytics do not merge until it passes.

2.6 Extension Blueprint and AI Assistance

Each artifact specified in this chapter is a template, not unrestricted, free-form code. This covers the connector files (Section 2.4), the analytics jobs (Section 2.5), and the test suites (Section 2.4.4, Section 2.5.3). This was an objective. The framework should be suited to AI assisted extension. The external requirements specification provides four Claude Code

skills that consume these templates and produce a reference implementation. The skills are organized by phase. `analyze-source` fetches the source's official API documentation and writes the connector page reference sections. `provision-source` emits the source-side runtime infrastructure as Terraform plus an install script. `generate-connector` produces the connector artifacts listed in Section 2.4 together with the Databricks-side runtime layout (secrets loader, install orchestrator, notebook entry wrappers where the category requires them, SQL bronze envelopes, multi-resource bundle fragments) and the runbook sections of the connector page. `validate-implementation` runs the test suite from Section 2.4.4 until the requirements from the specification are mapped to passing tests. A per-source data file at `src/connectors/<source>/operational.yml` records deployment-specific values such as cloud account, secret scope name, and target bronze table; missing fields are gathered conversationally during a skill run, so the operator does not have to know the schema upfront.

It was a conscious design decision that the set consists of four skills rather than a dedicated skill for every combination of role and application security category. This follows the recommended pattern for skills that span multiple variants of the same task (Anthropic, 2026). Each `SKILL.md` carries the role wide procedure. Guidance per category lives one level deeper, in a `references/<category>.md` file that the agent reads on demand. The seven application security categories are `cmdb`, `scm`, `sast`, `sca`, `secret`, `dast`, and `waf`.

This structure inherits two properties from the underlying skills mechanism (Agent Skills working group, 2026; Anthropic, 2025).The metadata overhead in the agent's system prompt remains fixed at four descriptions, regardless of how many categories are eventually added to the framework. The procedure is authored once and shared across categories. Chapter 3 reports on the execution of the catalog against the selected sources. Manually writing connectors is mundane. The approach of this thesis relies on modern AI assistance. A well specified framework plus a compact skill set makes the platform reproducibly constructible, with full traceability from test to requirement directly in the specification.

3. MVP Implementation

This chapter reports the reference implementation of the framework defined in Chapter 2. The deliverable is a Databricks Asset Bundle (Databricks, 2025d) that instantiates the three ingestion categories plus the artifact path carve out against the nine sources selected in Section 1.9: two SCM platforms, four scanners across static and dynamic testing, a CMDB, a secrets tool, and a runtime security source. It is a minimum viable product. Open items are listed in Section 3.5.

The contribution is twofold. The contract of three categories holds across all nine sources, and the resulting code fits one small repeating module layout. The chain of four skills ran end to end against every source, with reviewer subagent cycles catching named defect classes (Section 3.6.3, Section 3.6.3). All implementation artifacts under `src/connectors/<source>/` are output of the chain for all nine sources: the eight-file core (configuration, ingest, transform, mapping, severity, status, bundle resources, tests), the source-side runtime under `runtime/`, the Databricks-side runtime layout (secrets loader, install orchestrator, notebook entry wrappers, SQL bronze envelopes, multi-resource bundle fragments), and the connector page runbook sections. Section 3.4 shows two ends of the category range in concrete code: ServiceNow on Lakeflow Connect and GitHub on PyGitHub (PyGithub Contributors, 2025).

3.1 Methodology

The implementation follows the procedure below for every source in Section 1.9:

1. **Resolve the ingestion category.** The source is matched against the three categories defined in Section 2.4.1. The evaluation order is fixed: Lakeflow Connect (Databricks, 2025b) first, then a maintained Python SDK, then dlt. Tools that only ship a CLI fall outside the axis of three categories and follow the artifact pattern at the CI/CD step described in Section 2.4.2. Section 3.3 carries out this step for all nine sources.
2. **Instantiate the module layout for the connector.** Every connector carries the same files under `src/connectors/<source>/`: `ingest.py`, `transform.py`, `mapping.yml`, `config.yml`, `severity.yml`, and `status.yml`. Bundle resources, tests, and the secret loading script are colocated with the connector code. For Lakeflow Connect sources, `ingest.py` is reduced to a module docstring and ingestion is declared in the bundle fragment as a `pipelines` resource with an `ingestion_definition`.
3. **Apply the mapping declaratively.** The `mapping.yml` file for the connector encodes the column expressions from bronze to silver given by the published mapping requirements. Severity and status lookups live in the connector folder as `severity.yml` and `status.yml`. `src/platform/silver.py` provides severity rank and status bucket normalization helpers and deduplication utilities shared across connectors. The `transform.py`

module builds the silver DataFrame field by field, calling the shared helpers. The current MVP builds the DataFrame in Python rather than applying the YAML declaratively. Section 3.6.3 carries the declarative applicator as a thread.

4. **Test in the layered strategy from Section 2.4.4.** Tests divide along the signature of the code they exercise. Row level logic in pure Python (parser logic, mapping application, High Water Mark (HWM) state, dedup key construction) runs locally against `list[dict]` fixtures with no `SparkSession`. DataFrame logic (transformations, envelope stamping, schema enforcement) runs through a session scoped Databricks Connect fixture that attaches to a remote workspace, so the test runtime matches the production runtime. Tests live under `src/connectors/<source>/tests/` and are linked to requirement IDs through `@pytest.mark.requirement` markers.
5. **Run the skill chain pass for the source.** The Claude Code skill chain (analyze-source, provision-source, generate-connector, validate-implementation) executes against the source and writes evidence rows to the operator docs site: an Implementation log entry per pass and a Validation table keyed by requirement IDs. Each skill pass is then reviewed by a separate subagent (one for spec compliance, one for code quality), whose findings are folded back as follow up commits before the next source moves forward. Section 3.6.3 records the defect classes the reviewer cycle caught.

Step 1 has a deterministic answer for every source in the sample. Step 2 has an invariant layout across categories. That invariance is what makes the framework instantiable by an AI pipeline. A generator that emits the layout for a new source does not need to branch on category beyond what `ingest.py` contains and which bundle resource kind the source requires. Step 5 makes that generation procedure auditable: the operator docs site at `https://vkraus.github.io/appsec-mvp/` carries the per source Implementation log and Validation table as the trail of evidence.

3.2 Project Structure

The reference implementation is distributed across two repositories that separate the thesis document from the MVP code.

- The thesis repository holds the LaTeX sources for this document.
- The `appsec-mvp` repository (`https://github.com/vkraus/appsec-mvp`) holds the reference implementation described in the remainder of this section. The operator docs site lives inside the same repository under `mkdocs/` and publishes at `https://vkraus.github.io/appsec-mvp/` via GitHub Pages. The published site is the operator facing reference and the trail of evidence for each connector.

3.2.1 Layering rule

The repository is organized into three install layers with no upward or sideways dependencies in setup code: platform, then connectors, then analytics. The platform layer must not pre-declare resources for any specific connector. No source schemas, volumes, connections, or secrets live in `src/platform/resources/`. Each connector owns its own bronze and silver schemas, secret loading script, and bundle resources. The analytics layer reads only the connector agnostic silver tables (`silver.findings`, `silver.repositories`, `silver.app_repo_mapping`, `silver.hwm`). The one ordering constraint is data level, not setup code level. An SCM connector (GitHub or GitLab) must run first at job time because non SCM connectors map findings onto repository entities populated by SCM. The bundle deploys all declared resources regardless of install order.

3.2.2 Component colocation

Inside the `appsec-mvp` repository, every component (the platform, each connector, the analytics layer) owns its code, its tests, its bundle resources, its scripts, and (for connectors that bring up source systems) an optional Terraform runtime. Adding a new source is one new folder under `src/connectors/`. There are no top level `tests/`, `config/`, `resources/`, `sql/`, or `infra/terraform/` directories. Source code 3.1 shows the realized layout with one connector of each category expanded.

Source code 3.1
Realized MVP project layout. One connector of each category is expanded. The others follow the same layout.

```
appsec-mvp/
|-- databricks.yml                          # bundle root: targets,
    variables
|-- pyproject.toml                          # Python package metadata
|-- mkdocs/                                 # operator docs site source
|-- examples/end-to-end-demo/               # demo across scanners
|-- src/
|   |-- platform/                           # framework primitives (shared
    at runtime)
|   |   |-- silver.py                       # severity/status normalization
    + dedup
|   |   |-- hwm.py, schemas.py, ...         # HWM state, StructTypes,
    helpers
|   |   |-- resources/{platform,bootstrap-job}.yml
|   |   |-- scripts/bootstrap.sh            # one time bootstrap after
    deploy
|   |   |-- sql/silver_tables.sql           # standard silver DDL
|   |   \-- tests/                          # unit tests for the primitives
|   |-- connectors/
|   |   |-- servicenow/                     # category: Lakeflow Connect
|   |   |   |-- ingest.py                   # contract wrapper; pipeline
    declared in bundle
|   |   |   |-- transform.py
```

```
|   |   |   |-- mapping.yml, config.yml
|   |   |   |-- severity.yml, status.yml
|   |   |   |-- resources/{schemas,connection,pipeline,job}.yml
|   |   |   |-- scripts/load-secrets.sh
|   |   |   |-- runtime/                    # Terraform for the source
    system or its supporting cloud (all 9)
|   |   |   \-- tests/
|   |   |-- github/                         # category: SDK (PyGitHub)
|   |   |   |-- ingest.py, transform.py
|   |   |   |-- mapping.yml, config.yml
|   |   |   |-- severity.yml, status.yml
|   |   |   |-- resources/{schemas,job}.yml
|   |   |   |-- scripts/load-secrets.sh
|   |   |   |-- runtime/
|   |   |   \-- tests/
|   |   \-- (gitlab, sonarqube, semgrep, owasp_zap, dependency_track,
|   |       trufflehog, aws_waf): same layout, each with its own runtime/
|   \-- analytics/                          # gold layer SQL + jobs
|       |-- resources/{schemas,job}.yml
|       \-- sql/, tests/
```

The bundle resources for each source are colocated: `src/connectors/<source>/resources/` contains `schemas.yml` and either `job.yml` (for SDK and dlt connectors) or `pipeline.yml` plus `connection.yml` for Lakeflow Connect connectors, with `job.yml` alongside for the downstream transform task. The bundle root includes everything via the glob `src/{platform,connectors/*,analytics}/resources/*.yml`. Schemas, volumes, and secret loading scripts move with the connector folder, so installing or removing a connector does not touch shared files.

3.3 Ingestion Category Assignment

Each source is resolved to one of the three ingestion categories defined in Section 2.4.1 before its connector module is written. The evaluation order is Lakeflow Connect, then SDK, then dlt. CLI only tools fall outside the three categories and follow the CI/CD step artifact pattern (Section 2.4.2). The full per source functional requirements live on the operator docs site, and the resolutions below cite the deciding factor only.

- **ServiceNow**: Lakeflow Connect. Supported natively by the platform (reference), so `ingest.py` carries only the contract wrapper and the bundle fragment declares the pipeline.
- **GitHub**: SDK via PyGitHub. Lakeflow Connect does not support it at MVP time, and PyGitHub covers the required endpoints, handles `Link` header pagination, and exposes rate limit status (reference).
- **GitLab**: SDK via python-gitlab (python-gitlab Contributors, 2025). No Lakeflow Connect support, and the official client handles keyset pagination transparently (reference).
- **SonarQube** (SonarSource, 2024): dlt. No first party Python SDK, and the `rest_api` source template that ships with dlt covers all requirements declaratively against the Web API (reference).

- **Semgrep**: artifact path. The free CLI writes a JSON report per commit from a CI/CD step, and `ingest.py` reads from `periodic/semgrep/` and `cicd/semgrep/` prefixes (reference).
- **Dependency-Track** (OWASP Foundation, 2024b): dlt. No first party Python SDK, and the published OpenAPI document configures dlt directly (reference).
- **TruffleHog** (Truffle Security, 2024): artifact path. CLI tool with no server API, same pattern as Semgrep (reference).
- **OWASP ZAP**: artifact path. `ingest.py` supports two trigger contexts (on demand against the daemon, and reads of CI/CD step artifacts) and dispatches both into the same bronze table (reference).
- **AWS WAF**: SDK via boto3 (Amazon Web Services, 2025). The connector ships two ingestion modes selected by `config.yml::ingestion_mode`. The preferred mode reads full request logs delivered by Kinesis Data Firehose to an S3 prefix through an autoloader read. The fallback mode calls `boto3.client("wafv2").get_sampled_requests` with IAM signed credentials and preserves the per record `Weight` field (reference).

Table 3.1 collects the nine assignments alongside the key functional requirement that drove each choice.

Across the nine sources, one resolves to Lakeflow Connect (ServiceNow), three to an SDK (GitHub, GitLab, AWS WAF), two to dlt (SonarQube, Dependency-Track), and three to the artifact path of Section 2.4.2 (Semgrep, TruffleHog, OWASP ZAP).

3.4 Representative Connectors

This section shows the realized structure of a connector at each end of the category range. ServiceNow represents the Lakeflow Connect case, in which `ingest.py` reduces to a thin contract wrapper and the bundle fragment carries the ingestion specification. GitHub represents the SDK case, in which `ingest.py` composes client calls into iterators over bronze records.

3.4.1 ServiceNow: Lakeflow Connect Pipeline

The ServiceNow connector performs no live Python ingestion. The `ingest.py` module carries a module docstring pointing at the bundle fragment under `src/connectors/servicenow/resources/pipeline.yml` (reproduced in Source code 3.2) and a thin framework contract wrapper (`ingest_contract`) that validates the credential set so a misconfigured bundle job fails fast. Lakeflow Connect performs the actual HTTP ingestion against ServiceNow. Authentication is carried by a Databricks Connection declared in the same folder under `resources/connection.yml` and referenced by name. The `objects` block enumerates the two CMDB tables the silver layer consumes and their destination tables in bronze. Schedule

Table 3.1
Ingestion category assignment across the nine selected sources, showing the category chosen for each source and the library or mechanism that instantiates it.

Source	Category	Binding	Decisive requirement
ServiceNow	Lakeflow Connect	platform managed	First party LFC supported source. Full contract met declaratively.
GitHub	SDK	PyGitHub	Mature Python SDK covers REST endpoints, `Link` header pagination, and rate limit state.
GitLab	SDK	python-gitlab	Project recognized Python client covers REST with keyset pagination.
SonarQube	dltool	`rest_api` source	No maintained Python SDK. REST only access with offset pagination and `updateDate` high water mark.
Semgrep	Artifact path	CI/CD to Volume	CLI tool with no HTTP API. Ingestion via CI/CD step pattern.
Dependency-Track	dltool	OpenAPI driven	No first party SDK. REST with offset pagination and `lastOccurrence` high water mark.
TruffleHog	Artifact path	CI/CD to Volume	CLI tool with no server API. Ingestion via CI/CD step pattern.
OWASP ZAP	Artifact path	CI/CD to Volume	Scan report JSON produced in CI/CD and uploaded to Unity Catalog Volume.
AWS WAF	SDK	boto3	Two modes: log stream from Kinesis Data Firehose to S3 (preferred), boto3 `GetSampledRequests` (fallback).

and target catalog come from bundle variables so the same fragment can promote across dev, staging, and prod.

Source code 3.2

src/connectors/servicenow/resources/pipeline.yml

```
resources:
  pipelines:
    servicenow_ingest:
      name: servicenow_ingest
      catalog: ${var.catalog}
      target: bronze_servicenow
      continuous: false
      schedule:
        quartz_cron_expression: "0 0 * * * ?"
        timezone_id: UTC
      ingestion_definition:
        connection_name: servicenow
        objects:
          - table:
              source_schema: default
              source_table: cmdb_ci_business_app
              destination_catalog: ${var.catalog}
              destination_schema: bronze_servicenow
              destination_table: business_applications
          - table:
              source_schema: default
              source_table: cmdb_rel_ci
              destination_catalog: ${var.catalog}
              destination_schema: bronze_servicenow
              destination_table: app_cis
```

The framework prescribed connector layout is preserved. `mapping.yml`, `config.yml`, `severity.yml`, `status.yml`, and `transform.py` continue to govern the transformation from bronze to silver. Only `ingest.py` changes form, from a Python module to a declaration embedded in the bundle. That is the trade the Lakeflow Connect category offers: no pagination, authentication, or retry code written by hand, in exchange for binding the connector to a platform maintained capability.

3.4.2 GitHub: PyGitHub SDK Module

The GitHub connector uses PyGitHub. The `ingest.py` module exposes three names: a `build_github_client` factory, an `ingest_contract` framework wrapper that demonstrates the documented traversal, and a `verify_webhook_signature` HMAC helper that handles webhook receipt outside SDK territory. The traversal calls `client.get_organization(org).get_repos()` and receives a `PaginatedList` of `Repository` instances. Each repository then exposes `repo.get_pulls(state, since=hwm_value)` and `repo.get_codescan_alerts()` for walking the finding streams. Source code 3.3 reproduces the client factory as the representative fragment.

Source code 3.3

src/connectors/github/ingest.py *(client factory shown)*

```
from github import Auth, Github, GithubRetry


def build_github_client(token: str) -> Github:
    """Authenticated Github client with retry tuned for
    secondary rate limits."""
    auth = Auth.Token(token)
    retry = GithubRetry(
        total=10, backoff_factor=2.0, secondary_rate_wait=60
    )
    return Github(auth=auth, retry=retry, per_page=100)
```

PyGitHub absorbs the cross cutting concerns of authentication, pagination, and retry. `GithubRetry`, a `urllib3.Retry` subclass shipped with the library, recognizes GitHub's secondary rate limit signal that arrives as HTTP 403 with a `Retry-After` header. A plain `Retry(status_forcelist=[429, 5xx])` does not cover that path. The incremental state contract from the framework (REQ-ING-HWM) maps onto a built in PyGitHub mechanism: resource accessors such as `Repository.get_pulls(state="closed", since=hwm_value)` accept a `since` parameter, so high water mark filtering pushes down to the API call instead of running client side. The page size is set to PyGitHub's documented maximum of 100 to minimize round trips.

The tests under `src/connectors/github/tests/` fake PyGitHub at the object graph level rather than at the HTTP level. A test fixture builds `MagicMock` instances modeled on PyGitHub's `Github`, `Organization`, `Repository`, and `PaginatedList` classes, each exposing the attributes the production code calls. The contract under test is the composition of SDK calls in the connector, not the HTTP behavior of the library.

3.5 Aggregate Results

All nine sources are delivered as working connectors under `src/connectors/<source>/` with the full file set, bundle resources, secret loading scripts, and unit tests. Each also ships a Terraform runtime under `runtime/` that brings up the source system or its supporting cloud infrastructure.

Each source page on the operator docs site carries an Implementation log (four rows: analyze, provision, generate, validate, each linking to the producing commit) and a Validation table keyed by requirement IDs that map to `@pytest.mark.requirement` markers. The cross source matrix at platform/reference/catalog aggregates these into one (requirement × source) view with PASS or N/A per cell. The ten requirement identifiers (`REQ-ING-AUTH`, `REQ-ING-PAG`, `REQ-ING-RL`, `REQ-ING-HWM`, `REQ-TRF-MAP`, `REQ-TRF-SEV`, `REQ-TRF-STS`, `REQ-TRF-TS`, `REQ-DQ`, `REQ-DEDUP`) structure the matrix and form the trail of evidence behind the AI claim defended

in Section 3.6.3.

At commit `a8165ad` the suite holds 29 test files and 288 test functions, running under the layered strategy of step 4. The tests live under `src/connectors/<source>/tests/` and `src/platform/tests/`.

3.6 Discussion

3.6.1 Contract of three categories

Every source in the sample resolved to exactly one of Lakeflow Connect, SDK, or dlt, with the three artifact path sources (Semgrep, TruffleHog, OWASP ZAP) falling into the carve out (Section 2.4.2). No source required a category the framework does not admit. The Lakeflow Connect and SDK cases produce visibly different layouts: ServiceNow has no Python ingestion file of substance, while GitHub has a small flat `ingest.py` composed of SDK calls. Both preserve the module layout prescribed by the framework, with platform lock in on one side and per source client knowledge on the other.

3.6.2 Declarative mapping

Connectors share severity and status normalization helpers in `src/platform/silver.py` and keep lookup tables as YAML alongside the connector code. The `transform.py` for each source currently builds the silver DataFrame field by field calling those helpers, with `mapping.yml` documenting the intended derivation. Aligning the MVP with the declarative mapping prescription of Section 2.4.3 is a Section 3.6.3 thread.

3.6.3 Iteration Summary

This subsection collects iterations caught by review. Each is classified against the acceptance criterion defined in Section 3.6.3: **framework gap** (a specification extension would have prevented the issue and would generalize), **source specific issue** (not generalizable beyond one source), or **environment setup** (setup or infrastructure outside framework scope). Earlier iterations before the structural redesign exposed three framework gaps: a hand written shared HTTP primitive that introduced an unsanctioned fourth ingestion category, a missing boundary validator on incremental state inputs, and a misconfigured retry policy on the GitHub client. The redesign closed all three by enforcing the layering rule of Section 3.2.1 and by binding the connector contract to the colocated test suite. The findings below are from the reviewer cycle after the redesign.

- **Layering boundary that the redesign closed.** The layout before the redesign mixed top level `tests/`, `config/`, `resources/`, `sql/`, and `infra/terraform/` folders, and the platform layer predeclared resources for specific connectors. This permitted exactly the failure mode that the unsanctioned fourth ingestion category had exposed. The redesign moved every component to its own folder, deleted the predeclared resources from the platform layer, and stated the layering rule explicitly. **Framework gap.** The framework now states three install layers with no upward or sideways setup code dependencies, and the data level SCM first ordering is recorded separately as a job time constraint.
- **Schema drift between SQL Data Definition Language (DDL) and PySpark types.** Splitting the `sql/` tree into per layer ownership exposed inconsistencies between the DDL for the silver tables and the `StructType` definitions in `src/platform/schemas.py`. The reviewer flagged the drift. Connector side writers for `silver.repositories` and `silver.app_repo_mapping` remain pending. **Framework gap.** The framework should hold a single source of truth for silver schemas. Section 3.6.3 carries the consolidation as a thread.
- **Secret scope mismatch in the GitHub ingest entry.** The post deploy bootstrap script defines a secret scope named `mvp-connectors`. The GitHub ingest entry point read from a scope named `appsec`, a residue from an earlier prototype. The reviewer caught the mismatch during the bootstrap script review. **Environment setup.** The fix aligned the entry point with the documented scope name.
- **Worktree pollution.** Several review cycles caught files that the redesign deleted reappearing on disk and being staged for inadvertent reintroduction (likely an editor or antivirus recreating them). Reviewers caught the pattern before bad commits landed. **Environment setup.** The pattern is not attributable to the framework.
- **Cron schedule collision between GitHub and SonarQube.** The GitHub and SonarQube connector jobs were originally scheduled with overlapping quartz cron expressions, so both jobs would dispatch on the same minute mark and contend for the shared cluster. The reviewer flagged the collision and the SonarQube schedule was offset to a non aligned minute. **Source specific issue.** The correction is specific to two job schedules and does not generalize.
- **IRSA OIDC trust policy gaps.** Migrating each connector Terraform runtime under `src/connectors/<source>/runtime/` exposed incomplete trust policy details on the IAM Roles for Service Accounts (IRSA) bindings. The reviewer required full trust policy declarations and explicit operator inputs for OIDC issuer URLs. **Environment setup.** The corrections concern AWS account setup and are documented per connector under the `runtime/README.md` file.

Two of the six findings after the redesign classify as framework gaps. Both name a specific consolidation the framework should hold (the layering rule, a single source of truth for silver schemas). Three classify as environment setup and one as a source specific issue. The reviewer cycle caught all six before the commits landed, which is the empirical evidence behind the claim that the skill chain plus reviewer pattern catches named defect classes. Section 3.6.3 lifts these findings to a defended claim about where the framework holds and

where it admits further specification.

Conclusion

Thesis Outcomes and Contributions

This thesis delivers the three contributions announced in the abstract, evaluated against the design science framework of Hevner et al. (2004) and the outcome evaluation guidance of Peffers et al. (2007). The first is a **requirements specification** for an application security data platform, motivated and grounded in Chapter 1. The specification text was published on the documentation site and attached to the thesis instead of placed in an appendix because of its length. The second is a **reusable and extensible framework** (Chapter 2) covering reference architecture, data model, and the medallion based pipeline (Strengholt, 2025) from source records to consumption. The third is a **reference implementation** (Chapter 3) on Databricks against nine selected data sources.

The methodology maps concretely to the work. The framework is the design artifact (Hevner: design as an artifact; Peffers: design and development), motivated by the application security data consolidation problem in Chapter 1 (Hevner: problem relevance; Peffers: identify the problem and define objectives). It is instantiated in Chapter 3 (Peffers: demonstration), and evaluated through the traceability matrix and the AI instantiability assessment in Section 3.6.3 (Hevner: design evaluation and research contributions; Peffers: evaluation). Rigor comes from tests bound to requirement identifiers and from the iteration summary (Hevner: research rigor and design as a search process). Communication runs through the thesis and the operator facing documentation site (Hevner: communication of research; Peffers: communication). Two items are partial: the AI evaluation criterion was settled after the first reviewer cycles, not declared in advance, and design alternatives are recorded only at the level of the iteration summary.

The framework is **reusable** in the concrete sense that Chapter 3 introduced no new connector level design decisions beyond which sources to onboard. It is **extensible** in the sense that onboarding a new source reduces to the five step procedure of Section 3.1: category resolution, module instantiation, declarative mapping, layered testing, and skill chain pass with reviewer supervision. ServiceNow on Lakeflow Connect and GitHub on PyGitHub are concrete examples of the same contract supporting different integration patterns.

Evaluation of AI Instantiability

Chapter 2 aimed the framework outputs at an AI coding agent. The original target was fully autonomous generation from the agent skill catalog. The MVP was produced by invocations of that catalog under reviewer subagent supervision (Subagent-Driven Development).

Acceptance criterion

A correction counts as a **framework gap** when extending the specification (schema pattern, connector contract from Section 2.4.1.3, or declarative artifact) would have produced the right output on first pass and the extension would generalize. Corrections tracing to **source specific issues** or **environment setup** (setup, authentication, infrastructure) do not count against the framework.

Empirical outcome

The four-skill chain ran end to end against all nine sources, producing every per-source connector greenfield. Section 3.6.3 enumerates the defects the reviewer cycle caught: a layering boundary (framework gap), schema drift between SQL DDL and PySpark types (framework gap), a secret scope mismatch (environment setup), worktree pollution (environment setup), a cron schedule collision (source specific issue), and IRSA OIDC trust policy gaps (environment setup). Two of six classify as framework gaps. Evidence lives in the per connector Implementation logs at `https://vkraus.github.io/appsec-mvp/`.

Claim defended

The integration layer of the framework, that is every per source connector that adapts a security tool to the lakehouse contract, is AI **instantiable** under skill catalog invocation. The reference implementation was generated end to end across nine sources by the four skill chain with reviewer subagent cycles catching defects. The platform layer the connectors run against and the analytics layer they feed are deliberately hand written: the platform is the singleton contract the skills depend on by name, and the analytics layer is the specific evidence the thesis presents. The contract, mapping, and module layout reduce new connector work to a five step procedure that converges with bounded supervision. It is not yet AI **autonomous**: every connector still required reviewer cycles, and the schema drift gap plus the open items below remain. The empirical evidence above was produced by a single operator (the author) acting as both implementer and reviewer of the evaluation, without inter rater reliability or an independent grader. Closing the open items, refining the skills until invocations are self sustaining, and replicating the evaluation under a controlled study with multiple operators would measure the distance from instantiable to autonomous. That measurement is the subject of Future Work.

Limitations

The MVP is a working reference rather than a production ready integration. It is **platform specific**: it runs on Databricks over AWS object storage, the only combination exercised end to end. **Live tenant exercise** is partial: of the nine connectors, ServiceNow, GitHub, SonarQube, Semgrep, and OWASP ZAP have been exercised against live source systems during the construction of this thesis. The remaining four (GitLab, Dependency-Track, TruffleHog, AWS WAF) pass their unit tests against fixtures but have not been exercised against a live tenant. The connector contract in Section 2.4 is written to be cloud and platform agnostic in principle, but it has not been ported to Snowflake, Microsoft Fabric, or an on premises platform, so the portability claim rests on contract design alone, with no second demonstrated implementation.

Reproducibility requires a Databricks workspace with Unity Catalog, an AWS account with an S3 bucket and federated credentials, a Lakeflow Connect entitlement, a ServiceNow developer instance, and a GitHub Personal Access Token (PAT). The Databricks Asset Bundle deploys every workspace resource the bundle format covers natively. The post deploy shell script `src/platform/scripts/bootstrap.sh` creates the secret scope container, Unity Catalog storage credential, and Unity Catalog external location. Per connector Terraform modules under `src/connectors/<source>/runtime/` provision source side cloud infrastructure. The submitted `mvp.zip` and `docs.zip` cannot reproduce the build without these entitlements.

Several **open implementation items** remain. Population of `silver.repositories` and `silver.app_repo_mapping` from the SCM and CMDB connectors is pending: the DDL exists, but the writer code is not wired up, so joining scanner findings to repositories is structurally supported and currently empty. The `src/analytics/` layer is scaffolding pointing at a placeholder SQL file. The GitLab connector ships placeholder `ingest_entry.py` and `transform_entry.py` notebook wrappers and needs full job orchestration; six of the nine connectors do not require notebook entry wrappers because their ingestion category does not call for them. Inherited error handling sharp edges remain in `bootstrap.sh` and the ServiceNow Terraform runtime, both flagged for hardening.

The **variation** of the selected sources is biased toward REST JSON APIs with token authentication (five of the nine: GitHub, GitLab, ServiceNow, SonarQube, Dependency-Track). AWS WAF consumes a log stream from Kinesis Data Firehose to S3 with SigV4 service credentials rather than a bearer token, OWASP ZAP follows an artifact pattern, and the two file based sources (Semgrep, TruffleHog) deliver via S3 instead of an interactive API. Source categories not demonstrated include SOAP, gRPC, syslog, Kafka event streams, binary telemetry, and push only webhook native sources. The contract is written to accommodate these, but evidence is absent on a category by category basis.

Future Work

Closing the remaining framework gap. The layering boundary gap was closed by the structural redesign. The schema drift gap remains open. A single source of truth for silver schemas resolving the drift between SQL DDL and PySpark `StructType` definitions is the precondition for any stronger AI autonomy claim.

Completing the declarative mapping applicator. Section 2.4.3 treats `mapping.yml` as the authoritative specification consumed by a generic applicator, but the MVP builds the silver DataFrame field by field in `transform.py`. Completing the applicator would make adding a source a configuration edit rather than a code edit.

Realizing the remaining prescribed silver tables. The framework prescribes 15 silver tables (Section 2.2.3). The MVP realizes four (`applications`, `repositories`, `findings`, `app_repo_mapping`). Future iterations should add the remaining entity tables (`teams`, `commits`, `pull_requests`, `pipeline_runs`, `dependencies`, `branch_policies`), the reference tables (`vulnerabilities` from the NVD, `epss_scores`, `kev_entries`), and the remaining relationship tables (`finding_cve_mapping`, `dedup_links`). Adding each is a connector level extension under the same schema patterns, not a framework change.

Finishing job orchestration and bootstrap hardening. The GitLab connector ships placeholder `ingest_entry.py` and `transform_entry.py` wrappers and needs full job orchestration. The `src/analytics/` layer needs a real implementation. The bootstrap script and ServiceNow runtime need the hardening pass named in Limitations.

Downstream analytics on the gold layer. Large Language Model assisted triage and correlation over the gold tables is a natural extension and would exercise the data model of Section 2.2 in the direction it was designed for.

References

Agent Skills working group. (2026). *Agent skills specification*. Retrieved April 25, 2026, from https://agentskills.io/specification (cit. on p. 58).

Amazon Web Services. (2025). *boto3: AWS SDK for Python*. Retrieved April 24, 2026, from https://boto3.amazonaws.com/v1/documentation/api/latest/index.html (cit. on p. 63).

Amazon Web Services. (2026). *AWS WAF developer guide: Getting information about requests with GetSampledRequests*. Retrieved April 19, 2026, from https://docs.aws.amazon.com/waf/latest/APIReference/API_GetSampledRequests.html (cit. on pp. 14, 32).

Anderson, R. (2020). *Security engineering: A guide to building dependable distributed systems* (3rd ed.). John Wiley & Sons. (Cit. on pp. 12, 21, 22).

Anthropic. (2025). *Equipping agents for the real world with agent skills*. Retrieved April 25, 2026, from https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills (cit. on p. 58).

Anthropic. (2026). *Skill authoring best practices*. Retrieved April 25, 2026, from https://platform.claude.com/docs/en/agents-and-tools/agent-skills/best-practices (cit. on p. 58).

Armbrust, M., Das, T., Sun, L., Yavuz, B., Zhu, S., Murthy, M., Torres, J., van Hovell, H., Ionescu, A., Łuszczak, A., Świtakowski, M., Szafrański, M., Li, X., Ueshin, T., Mokhtar, M., Boncz, P., Ghodsi, A., Paranjpye, S., Senster, P., … Zaharia, M. (2020). Delta Lake: High-Performance ACID table storage over cloud object stores. *Proceedings of the VLDB Endowment*, *13*(12), 3411–3424. https://doi.org/10.14778/3415478.3415560 (cit. on p. 35).

Armbrust, M., Ghodsi, A., Xin, R., & Zaharia, M. (2021). Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics. *Proceedings of the 11th Annual Conference on Innovative Data Systems Research (CIDR)* (cit. on pp. 25, 35).

AXELOS. (2019). *ITIL foundation: ITIL 4 edition*. TSO (The Stationery Office). (Cit. on pp. 12, 17, 19).

Chacon, S., & Straub, B. (2014). *Pro git* (2nd ed.). Apress. (Cit. on p. 19).

Chess, B., & West, J. (2007). *Secure programming with static analysis*. Addison-Wesley. (Cit. on pp. 12, 22).

Cybersecurity and Infrastructure Security Agency. (2024). *Known exploited vulnerabilities catalog*. Retrieved April 11, 2026, from https://www.cisa.gov/known-exploited-vulnerabilities-catalog (cit. on p. 21).

DAMA International. (2017). *DAMA-DMBOK: Data management body of knowledge* (2nd ed.). Technics Publications. (Cit. on pp. 25, 26, 52).

Databricks. (2025a). *Databricks Feature Engineering*. Retrieved April 11, 2026, from https://docs.databricks.com/aws/en/machine-learning/feature-store/ (cit. on p. 56).

Databricks. (2025b). *Lakeflow Connect*. Retrieved April 24, 2026, from https://docs.databricks.com/aws/en/ingestion/lakeflow-connect/ (cit. on pp. 26, 59).

Databricks. (2025c). *Lakeflow Spark Declarative Pipelines*. Retrieved April 11, 2026, from https://docs.databricks.com/aws/en/ldp/ (cit. on pp. 14, 31, 35).

Databricks. (2025d). *What are Databricks Asset Bundles?* Retrieved April 11, 2026, from https://docs.databricks.com/aws/en/dev-tools/bundles (cit. on pp. 45, 59).

Databricks. (2025e). *What is Unity Catalog?* Retrieved April 11, 2026, from https://docs.databricks.com/aws/en/data-governance/unity-catalog/ (cit. on pp. 14, 35).

Databricks. (2026a). *Databricks enters security market with launch of Lakewatch: New open, agentic SIEM*. Retrieved April 11, 2026, from https://www.databricks.com/company/newsroom/press-releases/databricks-enters-security-market-launch-lakewatch-new-open-agentic (cit. on pp. 13, 29).

Databricks. (2026b). *Databricks Lakebase is now generally available*. Retrieved April 11, 2026, from https://www.databricks.com/blog/databricks-lakebase-generally-available (cit. on pp. 35, 56).

Databricks. (2026c). *Lakeflow Jobs*. Retrieved April 11, 2026, from https://docs.databricks.com/aws/en/jobs/ (cit. on p. 47).

Databricks. (2026d). *Observability in Databricks for jobs, Lakeflow Spark Declarative Pipelines, and Lakeflow Connect*. Retrieved April 11, 2026, from https://docs.databricks.com/aws/en/data-engineering/observability-best-practices (cit. on p. 47).

Databricks Labs. (2026). *Lakeflow Community Connectors*. Retrieved April 26, 2026, from https://github.com/databrickslabs/lakeflow-community-connectors (cit. on p. 26).

DefectDojo. (2024). *DefectDojo documentation*. Retrieved March 6, 2026, from https://documentation.defectdojo.com/ (cit. on pp. 13, 29).

Forsgren, N., Humble, J., & Kim, G. (2018). *Accelerate: The science of lean software and DevOps*. IT Revolution Press. (Cit. on pp. 20, 45).

Forum of Incident Response and Security Teams. (2019). *Common vulnerability scoring system v3.1: Specification document*. Retrieved March 6, 2026, from https://www.first.org/cvss/v3.1/specification-document (cit. on p. 21).

Gardner, D., Zumerle, D., & Bhat, M. (2023). *Innovation insight for application security posture management*. Gartner. Retrieved April 11, 2026, from https://www.gartner.com/en/documents/4326999 (cit. on pp. 12, 13, 29).

Gartner. (2024). Market Share: IT Service Management, Worldwide, 2024 [Gartner subscriber content, accessed via institutional subscription. Figure cited from the ServiceNow summary at https://www.servicenow.com/blogs/2025/no-1-6-tech-workflow-segments]. (Cit. on p. 17).

GitHub. (2024a). *GitHub REST API documentation*. Retrieved April 11, 2026, from https://docs.github.com/en/rest (cit. on pp. 20, 23).

GitHub. (2024b). *Rate limits for the REST API*. Retrieved April 18, 2026, from https://docs.github.com/en/rest/using-the-rest-api/rate-limits-for-the-rest-api (cit. on p. 20).

GitHub. (2025a). *About GitHub Advanced Security*. Retrieved April 11, 2026, from https://docs.github.com/en/get-started/learning-about-github/about-github-advanced-security (cit. on pp. 12, 29).

GitHub. (2025b). *Octoverse 2025: The state of open source*. Retrieved April 11, 2026, from https://octoverse.github.com/ (cit. on p. 19).

GitLab. (2025). *Application security testing*. Retrieved April 11, 2026, from https://docs.gitlab.com/user/application_security/ (cit. on p. 29).

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, *28*(1), 75–105. https://doi.org/10.2307/25148625 (cit. on pp. 15, 70).

Howard, M., & LeBlanc, D. (2006). *Writing secure code* (2nd ed.). Microsoft Press. (Cit. on p. 12).

Inmon, W. H. (2005). *Building the data warehouse* (4th ed.). John Wiley & Sons. (Cit. on pp. 25, 41).

Jacobs, J., Romanosky, S., Adjerid, I., & Baker, W. (2020). Improving vulnerability remediation through better exploit prediction. *Journal of Cybersecurity*, *6*(1), tyaa015. https://doi.org/10.1093/cybsec/tyaa015 (cit. on pp. 12, 21, 56).

JetBrains. (2025). *The state of CI/CD in 2025*. Retrieved April 11, 2026, from https://blog.jetbrains.com/teamcity/2025/10/the-state-of-cicd/ (cit. on p. 20).

Kim, G., Humble, J., Debois, P., Willis, J., & Forsgren, N. (2021). *The DevOps handbook: How to create world-class agility, reliability, & security in technology organizations* (2nd ed.). IT Revolution Press. (Cit. on pp. 21, 45).

Kimball, R., & Ross, M. (2013). *The data warehouse toolkit: The definitive guide to dimensional modeling* (3rd ed.). John Wiley & Sons. (Cit. on pp. 25, 28, 41, 42, 44).

Lee, D., Wentling, T., Haines, S., & Babu, P. (2024). *Delta Lake: The definitive guide: Modern data lakehouse architectures with data lakes*. O'Reilly Media. (Cit. on pp. 14, 25, 26, 35).

Linux Foundation. (2024). *SPDX specification*. Retrieved March 6, 2026, from https://spdx.dev/specifications/ (cit. on pp. 21, 22).

Mack, S. D. (2023). *The DevSecOps playbook: Deliver continuous security at speed*. John Wiley & Sons. (Cit. on p. 21).

McGraw, G. (2006). *Software security: Building security in*. Addison-Wesley. (Cit. on pp. 12, 21).

Miloslavskaya, N., & Tolstoy, A. (2016). Big data, fast data and data lake concepts. *Procedia Computer Science*, *88*, 300–305 (cit. on p. 25).

MITRE Corporation. (2024a). *CVE program*. Retrieved March 6, 2026, from https://www.cve.org/ (cit. on p. 21).

MITRE Corporation. (2024b). *CWE – common weakness enumeration*. Retrieved April 11, 2026, from https://cwe.mitre.org/ (cit. on pp. 21, 23).

MLflow. (2025). *MLflow documentation*. Retrieved April 11, 2026, from https://mlflow.org/docs/latest/ (cit. on pp. 56, 57).

National Institute of Standards and Technology. (2004). *Standards for security categorization of federal information and information systems* (tech. rep. No. FIPS PUB 199). U.S. Department of Commerce. Retrieved April 18, 2026, from https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.199.pdf (cit. on p. 17).

National Telecommunications and Information Administration. (2021). *The minimum elements for a software bill of materials (SBOM)*. Retrieved March 6, 2026, from https:

//www.ntia.gov/sites/default/files/publications/sbom_minimum_elements_report_0.pdf (cit. on p. 22).

OASIS Open. (2024). *Static analysis results interchange format (SARIF) version 2.1.0*. Retrieved March 6, 2026, from https://docs.oasis-open.org/sarif/sarif/v2.1.0/sarif-v2.1.0.html (cit. on p. 21).

OCSF Project. (2024). *Open cybersecurity schema framework*. Retrieved March 6, 2026, from https://schema.ocsf.io/ (cit. on pp. 21, 29).

Ohm, M., Plate, H., Sykosch, A., & Meier, M. (2020). Backstabber's knife collection: A review of open source software supply chain attacks. *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, 23–43. https://doi.org/10.1007/978-3-030-52683-2_2 (cit. on pp. 12, 22).

OpenSSF. (2024). *Supply-chain levels for software artifacts (SLSA)*. Retrieved March 6, 2026, from https://slsa.dev/ (cit. on pp. 12, 23).

OWASP Foundation. (2021). *OWASP top 10:2021*. Retrieved March 6, 2026, from https://owasp.org/Top10/ (cit. on p. 21).

OWASP Foundation. (2024a). *CycloneDX bill of materials standard*. Retrieved March 6, 2026, from https://cyclonedx.org/ (cit. on pp. 21, 22).

OWASP Foundation. (2024b). *OWASP Dependency-Track documentation*. Retrieved April 11, 2026, from https://docs.dependencytrack.org/ (cit. on p. 63).

OWASP Foundation. (2024c). *OWASP ModSecurity core rule set*. Retrieved April 18, 2026, from https://coreruleset.org/ (cit. on p. 24).

OWASP Foundation. (2024d). *OWASP ZAP documentation*. Retrieved April 11, 2026, from https://www.zaproxy.org/docs/ (cit. on pp. 14, 32).

Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, *24*(3), 45–77. https://doi.org/10.2753/MIS0742-1222240302 (cit. on pp. 15, 70).

Pressman, R. S., & Maxim, B. R. (2019). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill. (Cit. on p. 19).

PyGithub Contributors. (2025). *PyGithub documentation*. Retrieved April 24, 2026, from https://pygithub.readthedocs.io/en/stable/ (cit. on p. 59).

python-gitlab Contributors. (2025). *python-gitlab: A Python wrapper for the GitLab API*. Retrieved April 24, 2026, from https://python-gitlab.readthedocs.io/en/stable/ (cit. on p. 62).

Reis, J., & Housley, M. (2022). *Fundamentals of data engineering: Plan and build robust data systems*. O'Reilly Media. (Cit. on pp. 25, 26, 42, 52).

Semgrep. (2024). *Semgrep documentation*. Retrieved April 11, 2026, from https://semgrep.dev/docs/ (cit. on p. 12).

ServiceNow. (2024a). *CMDB instance API reference*. Retrieved April 18, 2026, from https://www.servicenow.com/docs/bundle/zurich-api-reference/page/integrate/inbound-rest/concept/cmdb-instance-api.html (cit. on p. 18).

ServiceNow. (2024b). *REST API authentication*. Retrieved April 18, 2026, from https://www.servicenow.com/docs/bundle/xanadu-api-reference/page/integrate/inbound-rest/concept/c_RESTAPI.html (cit. on p. 18).

ServiceNow. (2024c). *Scripted REST APIs*. Retrieved April 18, 2026, from https://www.servicenow.com/docs/bundle/zurich-api-reference/page/integrate/custom-web-services/concept/c_CustomWebServices.html (cit. on p. 18).

ServiceNow. (2024d). *Table API reference*. Retrieved April 18, 2026, from https://www.servicenow.com/docs/bundle/zurich-api-reference/page/integrate/inbound-rest/concept/c_TableAPI.html (cit. on p. 18).

Skoulikari, A. (2024). *Learning git: A hands-on and visual guide to the basics of git*. O'Reilly Media. (Cit. on p. 19).

Snyk. (2024). *Snyk API documentation*. Retrieved April 11, 2026, from https://docs.snyk.io/snyk-api (cit. on p. 12).

Sommerville, I. (2015). *Software engineering* (10th ed.). Pearson. (Cit. on p. 19).

SonarSource. (2024). *SonarQube web API documentation*. Retrieved April 11, 2026, from https://docs.sonarsource.com/sonarqube-server/latest/extension-guide/web-api/ (cit. on p. 62).

Stack Overflow. (2025). *2025 stack overflow developer survey*. Retrieved April 11, 2026, from https://survey.stackoverflow.co/2025/ (cit. on pp. 19, 20).

Stallings, W., & Brown, L. (2024). *Computer security: Principles and practice* (5th ed.). Pearson. (Cit. on pp. 12, 17, 21).

Strengholt, P. (2025). *Building medallion architectures: Designing with Delta Lake and Spark*. O'Reilly Media. (Cit. on pp. 14, 25, 31, 70).

Stuttard, D., & Pinto, M. (2011). *The web application hacker's handbook: Finding and exploiting security flaws* (2nd ed.). John Wiley & Sons. (Cit. on pp. 12, 23).

Thalpati, G. A. (2024). *Practical lakehouse architecture: Designing and implementing modern data platforms at scale*. Apress. (Cit. on p. 25).

Truffle Security. (2024). *TruffleHog documentation*. Retrieved April 11, 2026, from https://trufflesecurity.com/trufflehog (cit. on p. 63).

Williams, R., & Gonzalez, K. (2021). *3 steps to improve IT service view CMDB data quality*. Retrieved April 11, 2026, from https://www.gartner.com/en/documents/3994127 (cit. on pp. 12, 19).

Winters, T., Manshreck, T., & Wright, H. (2020). *Software engineering at Google: Lessons learned from programming over time*. O'Reilly Media. (Cit. on p. 19).

Appendices

A. Generative AI Use Disclosure

This appendix expands the Declaration on the Use of Artificial Intelligence from the front matter of this thesis. The five sections below disclose each use in the same order as the front matter declaration. The central use is the AI assisted construction of the reference implementation (Chapter 3), which is evaluated in Section 3.6.3 rather than treated as incidental assistance.

The grammar and language style assistance described in the first section below used the Overleaf AI assistant on the thesis manuscript. All other AI assistance reported in this appendix used Claude models (Anthropic) accessed through the Claude Code CLI, with model versions spanning Claude Opus 4.5 through Claude Opus 4.7 over the drafting period of the thesis.

A.1 Text Editing

I did not rely on AI assistants to autonomously generate thesis text. I manually wrote all new text material in Overleaf editor, and only afterward revised it with AI support.

AI assistance feature in Overleaf online editor was used for grammar checking, style refinement, synonym replacement, sentence rephrasing, and other editing of the content I authored. I relied on the assistant throughout my drafts to identify errors, tighten phrasing, and highlight inconsistencies. I accepted, rejected, or revised the suggestions.

Several review cycles were carried out across the entire thesis using the Claude Code tool with Claude Opus models to condense it to its target length. These iterations rephrased longer passages into more concise wording and relocated some details into the attached product documentation, which contains the requirements specification and reference material per data source. The restructuring decisions were mine. The AI assistant proposed cuts and rephrasing that I accepted, rejected, or revised.

The research question, chapter structure, major contributions, and central claims are all mine. Positioning against prior work (Section 1.8) and the conceptual domain model (Section 1.7.3) were also authored by me. Literature selection is mine, Claude Code was used to generate most bibliography entries in LaTeX format. Several entries point at vendor product pages and white papers (used as primary sources for product capabilities and feature claims) instead of peer reviewed literature. These are flagged inline with the specific claim each one supports.

A.2 Images

AI assistant generated TikZ figures from my sketch or textual description of the content and layout, and together we iterated them for correctness and readability.

A.3 Source Code (`mvp.zip`)

The reference implementation in Chapter 3 (submitted as `mvp.zip`) was built across three distinct AI assisted workflows, all using Claude Code with Anthropic Claude Opus models.

The first workflow predates the public `appsec-mvp` repository. Four connectors (ServiceNow, GitHub, Semgrep, OWASP ZAP) were hand coded by me during the analysis phase, with Claude Code used conversationally for boilerplate and refactoring, and committed as the initial repository snapshot. They have since been re-emitted greenfield by the skill chain. The current code in the repository is output of the chain for all nine connectors.

The second workflow was the connector skill chain published in the documentation site (Product Documentation section below). It ran end to end against all nine selected data sources, producing every per source connector greenfield. Each pass wrote evidence rows to the product documentation (an Implementation log entry and a Validation table keyed by requirement IDs).

The third workflow was the structural redesign that centered the implementation on Databricks, run on a separate branch and merged through pull request #2. It applied the Subagent-Driven Development pattern from the Anthropic Superpowers plugin (brainstorm, then spec, then plan, then execute) using artifacts under `docs/superpowers/{specs,plans}/` in the `appsec-mvp` repository (the thesis repository keeps a local `docs/superpowers/specs/` for review notes only). Each task was implemented by a fresh Opus subagent and then reviewed by a separate specification compliance subagent and a separate code quality subagent across two reviewer cycles. The reviewer cycles caught the defect classes recorded in Section 3.6.3: a layering boundary, schema drift between SQL DDL and PySpark types, a secret scope mismatch, worktree pollution, a cron schedule collision, and IRSA OIDC trust policy gaps. The same Subagent-Driven pattern with paired reviewers was also applied to skill chain runs in the second workflow.

This is my main AI use. Section 3.6.3 evaluates it. The four skills published on the documentation site (`analyze-source`, `provision-source`, `generate-connector`, `validate-implementation`) are an AI assisted companion to the requirements specification.

A.4 Product Documentation (`docs.zip`)

The product documentation, attached as `docs.zip`, is AI assisted content.

I authored the four skills on the skills page in Claude Code, using the Anthropic skill writing skill. Each skill carries one `SKILL.md` for the role and one `references/<category>.md` for each application security category exercised by the MVP (CMDB, SCM, SAST, SCA, Secrets, DAST, WAF), totaling 32 authored files across the four skills. Container scanning and IaC scanning are part of the framework finding categories enumerated at Section 2.2.3.2 but are not exercised by any MVP connector, so the skill catalog does not yet ship reference pages for them. The layout follows the published Anthropic recommendation for skills that span multiple variants, defended in the Extension Blueprint section of Chapter 2. I iterated each skill until it produced the intended output. The skill chain was then used to generate connector code for the reference implementation, as disclosed in the Source Code section above.

A.5 Supporting Tools

I used AI assistance in Claude Code for maintaining LaTeX script files that were used to build this document.